JTS GO

JAY TECH SOLUTIONS

The JTS GO Programming Language

From Basics to Machine Learning &
Web Development

COMPREHENSIVE UNIVERSITY-LEVEL EDITION

VERSION 2.1

LANGUAGE FUNDAMENTALS · BUILT-IN FUNCTION REFERENCE · STANDARD LIBRARY (SCROLLS)
OBJECT-ORIENTED PROGRAMMING · MACHINE LEARNING · WEB DEVELOPMENT

ASWINJAY — CEO, JAY TECH SOLUTIONS

The JTS GO Programming Language

A Complete Reference & Learning Guide

Version 2.1 · Comprehensive University-Level Edition

ASWINJAY

CEO, Jay Tech Solutions

The JTS GO Programming Language

A Complete Reference & Learning Guide

First Edition · Comprehensive University-Level Edition · 2026

Every example in this book has been executed and verified against the official JTS GO interpreter (Version 2.1). Example outputs reflect actual program output unless otherwise noted.

Published by Jay Tech Solutions
https://github.com/jayaswinjay-web/jtsdk

Preface

Programming languages shape the way we think about problems. Some languages ask you to master decades of boilerplate before you can write your first useful program; others exist to make powerful ideas *simple*. JTS GO belongs to the second family. It is a modern, dynamically typed, bytecode-compiled language with a clean Python-inspired syntax, designed to be the easiest programming language to learn while still being powerful enough for real-world software: object-oriented design, web servers, file processing, and machine learning.

This book is a complete learning guide and reference. It is written for students at the university level as well as self-learners, and it is organised the way great computer science textbooks are: we begin with the smallest building blocks — syntax, values, and operators — and build upward through control flow, functions, object-oriented programming, and the standard library, until we reach advanced topics such as web development and neural networks.

Every concept is accompanied by complete, runnable examples. Wherever possible, the exact output of each example is shown, because a programming book that shows you code without showing you what the code produces is only half useful. All examples have been verified against the official JTS GO interpreter, Version 2.1.

JTS GO is a young language with an enthusiastic community. Its built-in functions, its scroll system (the standard library), and its language features are evolving. The version described in this book is Version 2.1, which introduced seven new standard-library scrolls and a suite of new string, sequence, statistical, and file utilities. Where the language may change in the future, this book teaches the *concepts*, which are far more durable than the details.

I hope this book becomes a trusted companion on your journey — whether you are writing your very first line of code, building your first web server, or training your first neural network. Welcome to JTS GO.

— Aswinjay
CEO, Jay Tech Solutions

Contents

1. Introduction to JTS GO

1.1 What is JTS GO?

JTS GO is a high-level, dynamically typed, general-purpose programming language developed by Jay Tech Solutions. It combines the readability of modern scripting languages with the speed of a compiled bytecode virtual machine. Programs written in JTS GO are compiled to bytecode and executed on the JTS Virtual Machine (JTS VM), giving you the productivity of an interpreted language and the structure of a compiled one.

Design Goal	How JTS GO Achieves It
Easy to learn	Python-inspired, indentation-based syntax with plain-English keywords
Fast execution	Source is compiled to bytecode, then run on a high-performance virtual machine
Object oriented	Classes, methods, inheritance, and the `self` keyword
Built-in ML/AI	Tensors, matrices, `sigmoid`, `relu`, and mean squared error
Web development	Built-in HTTP server and HTTP request functions
Zero boilerplate	No imports required for the core language; no type declarations

1.2 A First Impression

Here is a complete JTS GO program:

```
# hello.jts — your first program
print("Hello, World!")
print("Welcome to JTS GO!")
```

```
Hello, World!
Welcome to JTS GO!
```

That is the entire program. There are no package declarations, no `public static void main`, no compiler directives — just one line that prints a message. This is the philosophy of JTS GO: remove everything that is not essential, and keep what matters.

1.3 The Design Philosophy

- **Clarity over cleverness.** A JTS GO program reads like English. Keywords are full words: `func`, `if`, `while`, `class`, `return`.

- **Indentation is structure.** Code blocks are shown by indentation and closed with the keyword `end`. There are no braces to balance and no semicolons to forget.
- **Dynamic typing.** Variables take on whatever type they are assigned. The same variable can hold a number, a string, a list, or a class instance over the course of a program.
- **Batteries included.** The core language ships with dozens of built-in functions — from string handling to machine learning — plus a standard library of *scrolls* that add statistics, text processing, file utilities, and date/time helpers.
- **One language, many domains.** The same language that teaches a beginner to print `Hello, World!` is used in this book to build an HTTP web server and to train a multi-layer neural network.

1.4 The JTS Toolchain

JTS GO uses a compile-then-run model. The flow is:

```
.jts source  →  compiler (jtsc)  →  .jbc bytecode  →  virtual machine (jtsvm)  →  output
```

For convenience, the single `jts` command performs both steps. This means your source files are checked for errors at compile time, and the resulting bytecode runs quickly and deterministically.

Command	Purpose
`jts file.jts`	Compile and run a program in one step
`jtsc file.jts`	Compile to bytecode only (produces a `.jbc` file)
`jtsvm file.jbc`	Run a pre-compiled bytecode file
`jts --update`	Update JTS GO to the latest version

1.5 What This Book Covers

1. **Part I — Getting Started:** installation, tools, and your first programs.
2. **Part II — Language Fundamentals:** syntax, types, variables, operators, control flow, functions, object-oriented programming, collections, strings, and error handling.
3. **Part III — Built-in Functions:** a complete, documented reference for every built-in function.
4. **Part IV — The Standard Library (Scrolls):** the scroll system and all fourteen standard scrolls.
5. **Part V — Advanced Topics:** machine learning, web development, debugging, and best practices.
6. **Part VI — Appendices:** keyword reference, operator reference, and quick reference tables.

1.6 About the Examples

Every code example in this book was executed with the official JTS GO interpreter, Version 2.1. When an example produces output, the output is shown immediately after the code in a shaded box labelled with a dark background. Example outputs are therefore trustworthy — if you run the same code on the same

version, you will see the same results, except where the behavior depends on the current date, time, time zone, or random values.

> **Tip:** The fastest way to learn JTS GO is to type each example yourself and then modify it. The language rewards experimentation: there is no boilerplate standing between you and a working program.

2. Installation and Setup

2.1 System Requirements

Requirement	Minimum
Operating system	Windows 10+, macOS 10.15+, or Linux (Ubuntu 18.04+)
Node.js	v14.0 or higher (only needed for the npm installation method)
Disk space	About 50 MB
RAM	256 MB minimum

2.2 Installation with npm (Recommended)

The easiest way to install JTS GO is through the Node Package Manager:

```
npm install -g @jaytechsolutions/jts-go
```

This installs three commands globally: `jts`, `jtsc`, and `jtsvm`. Verify the installation by printing the version:

```
jts --version
```

2.3 Manual Installation

If you prefer not to use npm, download the latest release archive from the JTS GO GitHub releases page and extract it to a folder such as `C:\jtsdk` on Windows or `/usr/local/jtsdk` on macOS/Linux. Then add the `bin` folder inside it to your system `PATH`.

Windows (PowerShell, as Administrator):

```
$currentPath = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$currentPath;C:\jtsdk\bin", "User")
```

macOS / Linux:

```
export PATH="$PATH:/usr/local/jtsdk/bin"
```

2.4 What Gets Installed

Installing the JTS SDK gives you this layout:

```
jtsdk/
├── bin/              # CLI executables (jts, jtsc, jtsvm)
├── docs/             # Language documentation
├── examples/         # Example programs
├── scrolls/          # The standard library (text, stats, seq, numbers, ...)
├── LICENSE
└── README.md
```

The `scrolls` folder deserves special attention. It contains the JTS GO standard library as plain, readable `.jts` source files. Because scrolls are ordinary JTS GO code, you can read every standard-library function and even copy its implementation into your own programs. Chapter 25 explains the scroll system in detail.

2.5 The JTS IDE

Jay Tech Solutions also ships a free desktop IDE for JTS GO. The IDE provides a source editor, syntax highlighting, one-click run, and a built-in console so that you can develop, test, and run programs without leaving a single window. The IDE installer is distributed together with the SDK.

> **Note:** None of the examples in this book require the IDE. Everything can be typed in any plain-text editor and run from a command line. Use whatever editor is comfortable for you.

2.6 Troubleshooting Installation

Symptom	Fix
`'jts' is not recognized`	The `bin` folder is not on `PATH`. Add it, or run the full path to the executable.
npm permission errors	Run the npm command from a terminal with administrator privileges, or configure a user-level global prefix.
`Cannot find module` during npm install	Update npm and Node.js to current versions, then retry.
The interpreter runs but scrolls are missing	Scrolls are loaded from the `scrolls` folder. Make sure it sits next to the interpreter, or set the `JTS_SCROLLS` environment variable to its location.

3. Your First Programs

3.1 Hello, World

Create a file named `hello.jts` containing:

```
print("Hello, World!")
print("Welcome to JTS GO!")
```

Run it from the command line:

```
jts hello.jts
```

```
Hello, World!
Welcome to JTS GO!
```

Congratulations — you have written and executed your first JTS GO program. Two things happened: the compiler translated your source into bytecode, and the virtual machine executed that bytecode.

3.2 Printing Many Kinds of Values

`print` accepts any value. Numbers, booleans, and `nil` are all printed naturally:

```
print(42)
print(3.14159)
print(true)
print(false)
print(nil)
```

```
42
3.14159
true
false
nil
```

3.3 Reading User Input

The `input` function reads a line of text from the user. It accepts an optional prompt:

```
name = input("What is your name? ")
print("Hello, " + name + "!")
```

If the user types `Alice`, the output is:

```
What is your name? Alice
Hello, Alice!
```

> **Note:** `input` always returns a string. If you need a number, you can convert it with `number()` or `int()` (Chapters 14 and 15).

3.4 A Simple Calculator

```
# A friendly calculator
a = input("Enter first number: ")
b = input("Enter second number: ")

print("You entered: " + a + " and " + b)
print("Their sum is: " + (number(a) + number(b)))
```

This example introduces three ideas we will study in depth: variables, string concatenation with the `+` operator, and the `number()` conversion function.

3.5 Separating Compilation from Execution

You normally run everything with `jts`, but the toolchain can also be used in two explicit steps. This is useful when you want to distribute a program without its source code:

```
# Step 1 — compile
jtsc hello.jts          # produces hello.jbc

# Step 2 — run
jtsvm hello.jbc
```

3.6 The Building Blocks Ahead

Every language has a small set of fundamental ideas. For JTS GO those ideas are:

- **Comments** — text the compiler ignores (Chapter 4).
- **Values and types** — numbers, strings, booleans, `nil`, lists, dicts, and sets (Chapters 5 and 11).
- **Variables** — named storage for values (Chapter 6).
- **Operators** — the vocabulary of expressions (Chapter 7).
- **Statements** — `if`, `while`, and `for` that control what runs and when (Chapter 8).
- **Functions** — reusable blocks of code (Chapter 9).
- **Classes** — blueprints for objects (Chapter 10).

Chapter 4 begins at the very beginning: the syntax and lexical structure of the language.

4. Syntax and Lexical Structure

4.1 Comments

Comments begin with the `#` character and run to the end of the line. The compiler ignores them. Use comments to explain *why* code exists, not just what it does.

```
# This is a comment
name = "JTS"          # this is an inline comment

# A multi-line comment is just several line comments
# like these three lines.
```

4.2 Code Blocks and the `end` Keyword

JTS GO uses indentation to show the body of a block, and every block is terminated by the keyword `end`. Blocks begin after `if`, `elif`, `else`, `while`, `for`, `func`, `class`, `try`, `catch`, and `finally`.

```
if temperature > 80
    print("It's hot!")
end
```

- Use four spaces (recommended) or tabs, but be **consistent** within a program.
- Every opened block must be closed with `end`.
- Blocks can be nested to any depth.

```
func check_age(age)
    if age >= 18
        print("Adult")
    else
        print("Minor")
    end
end
```

> **Common error:** forgetting an `end`. The compiler reports `Expect 'end' after block` (or similar). When you see this error, check the most recent block that has not been closed.

4.3 Statements and Line Structure

Each statement occupies its own line. A line break terminates a statement, so there are no semicolons. A few points worth knowing:

- Blank lines and extra spaces are allowed anywhere and make code more readable.
- Indentation is meaningful only as the opening of a block — the compiler uses indentation levels (the *INDENT* and *DEDENT* tokens) together with `end`.
- Expressions may be wrapped in parentheses, which is especially useful when building a long concatenation:

```
a = 10
b = 3
print("a + b = " + (a + b))            # a + b = 13
print("(a + b) * 2 = " + ((a + b) * 2))  # (a + b) * 2 = 26
```

4.4 Literals

Literals are the values you write directly in source code.

Literal kind	Examples
Integer numbers	`0`, `42`, `-17`
Floating-point numbers	`3.14`, `-0.5`, `1e3`
Strings (double quotes)	`"hello"`, `""`
Booleans	`true`, `false`
Nil	`nil`
Lists	`[1, 2, 3]`, `[]`, `["a", 42, true]`
Dicts	`{"name": "JTS", "year": 2026}`
Sets	`{1, 2, 3}`, `{}` is an empty dict, use `set([])` for an empty set

Set-literal quirk: a brace literal with **two or more** elements is a set (`{1, 2}`), while a brace literal with **one** element is treated as a dict and fails (`{1}` is an error: `Expect string or identifier as dict key`). Use the `set` function — for example `set([1])` — when you need a single-element set. Chapter 11 covers this in detail.

4.5 Identifiers

Variable, function, and class names are called identifiers. The rules are simple:

- Names contain letters, digits, and underscores.
- Names must start with a letter or an underscore.
- Names are case-sensitive: `age`, `Age`, and `AGE` are three different variables.
- There is no reserved type-declaration syntax; assignment creates a variable (Chapter 6).

```
# Valid names
count = 10
_user_name = "admin"
max_score = 100
item2 = "second"

# Invalid names (these cause compile errors)
# 2item = "bad"      # cannot start with a digit
# my-var = "bad"     # hyphen is not allowed
# my var = "bad"     # spaces are not allowed
```

4.6 Keywords

Keywords are words with a fixed meaning in the language; you cannot use them as identifiers. The complete list is covered in Appendix A. The core keywords are:

```
and or not if elif else end while for in to func return
true false nil class self super extends new is del assert
try catch throw finally yield break continue import append
len print input type str int float bool number matrix tensor
set var list
```

Some of these words (for example `print`, `len`, `int`) are also available as built-in functions. Do not create variables with these names.

4.7 The Structure of a Program

A JTS GO program is a sequence of statements executed top to bottom. Definitions of functions and classes can appear in any order, but a function must be *called* after it has been defined. The conventional structure of a larger program is:

```
# 1. Bring standard-library scrolls (Chapter 25)
bring math
bring stats

# 2. Global constants and configuration
app_name = "Weather Station"
version = "2.1"

# 3. Helper functions
func kelvin_to_celsius(k)
    return k - 273.15
end

# 4. The main logic, called at the end
func main()
    print(app_name + " v" + version)
    print(kelvin_to_celsius(300))
end

main()
```

4.8 Case Study: A Complete Small Program

```
# A simple greeting program using every element so far
print("Welcome to JTS GO!")

name = input("What is your name? ")
name_length = len(name)

if name_length > 0
    print("Hello, " + name + "!")
    print("Your name has " + name_length + " characters.")
else
    print("You didn't enter a name!")
end
```

This program uses a comment, a literal, a built-in function, a variable, string concatenation, and an `if/else` block — a good summary of everything in this chapter.

5. Data Types

5.1 Types Are Dynamic

JTS GO is dynamically typed. You never declare a variable's type; the type is determined automatically from the value you assign, and it can change later. This keeps the syntax clean while giving you enormous flexibility.

5.2 The Core Types

Type	Description	Example
`number`	Integers and floating-point numbers	`42`, `3.14`, `-7`
`string`	Text enclosed in double quotes	`"hello"`, `""`
`boolean`	Logical truth values	`true`, `false`
`nil`	The absence of a value	`nil`
`list`	An ordered collection	`[1, 2, 3]`
`dict`	A mapping from string keys to values	`{"a": 1}`
`set`	An unordered collection of unique values	`{1, 2, 3}`
`function`	A callable defined with `func`	`func f() end`
`class` / instance	Object-oriented types	`new Animal("Cat")`

5.3 Numbers

There is a single number type that covers both integers and floating-point values. Arithmetic behaves as you expect from mathematics:

```
a = 10
b = 3

print(a + b)     # 13
print(a - b)     # 7
print(a * b)     # 30
print(a / b)     # 3.33333
print(a % b)     # 1   (the remainder)
print(-a)        # -10 (unary negation)
```

```
13
7
30
3.33333
1
-10
```

Floor division `//`

In addition to the five arithmetic operators above, JTS GO provides floor division, which divides and rounds down to the nearest integer:

```
print(10 // 3)     # 3
print(-10 // 3)    # -4  (rounds toward negative infinity)
print(10 // 2)     # 5
```

Floor division is used throughout the standard library, for example to find the middle index of a list.

Exponentiation `**`

```
print(2 ** 10)     # 1024
print(3 ** 2)      # 9
print(4 ** 0.5)    # 2
```

Operator precedence

Multiplication and division bind more tightly than addition and subtraction, exactly as in standard mathematics. Parentheses override the natural precedence.

```
print(2 + 3 * 4)       # 14  (not 20)
print((2 + 3) * 4)     # 20
```

5.4 Strings

Strings are sequences of characters written in double quotes:

```
name = "JTS GO"
empty = ""
greeting = "Hello, World!"
```

String length

```
print(len("JTS"))       # 3
print(len(""))          # 0
print(len("hello"))     # 5
```

Concatenation with `+`

```
first = "Hello"
second = "World"
result = first + " " + second
print(result)        # Hello World
```

Automatic conversion in concatenation

When you add a number to a string, the number is converted to its string form automatically:

```
age = 25
print("Age: " + age)          # Age: 25
price = 19.99
print("Price: $" + price)     # Price: $19.99
```

> **Note:** Concatenating a string with a list, dict, or set also converts that value to its printed representation, which is useful when debugging (Chapter 42).

Indexing individual characters

Individual characters can be read with square brackets; indices start at 0. Negative indices count from the end.

```
s = "Hello"
print(s[0])    # H
print(s[4])    # o
print(s[-1])   # o
print(s[-2])   # l
```

5.5 Booleans

Booleans are the values `true` and `false`. They are produced by comparison and logical operators and used to drive control flow (Chapter 8).

```
is_active = true
age = 25
is_adult = age >= 18
print(is_adult)    # true
```

5.6 Nil

`nil` represents the absence of a value — similar to `null` or `None` in other languages. Functions that do not return a value implicitly return `nil`. Indexing a dict with a missing key yields `nil` (Chapter 11).

```
result = nil
print(result)          # nil
print(type(result))    # nil
```

5.7 Checking Types with `type()`

The built-in `type()` function returns the type of a value as a string:

```
print(type(42))          # number
print(type(3.14))        # number
print(type("hello"))     # string
print(type(true))        # boolean
print(type(nil))         # nil
print(type([1, 2]))      # list
print(type({"a": 1}))    # dict
print(type({1, 2}))      # set
```

Because the result is a string, it can be compared in conditions:

```
value = "hello"
if type(value) == "string"
    print("value is a string")
end
```

5.8 Truthiness

When a value is used as a condition, it is evaluated as either truthy or falsy:

Falsy	Truthy
`false`	`true`
`nil`	Any non-zero number
`0`	Any non-empty string

"" (empty string)	Any list, dict, or set (even empty ones)

```
if 0
    print("This will NOT print — 0 is falsy")
end

if 1
    print("This WILL print — 1 is truthy")
end

if ""
    print("This will NOT print — empty string is falsy")
end

if "hello"
    print("This WILL print — non-empty string is truthy")
end
```

```
This WILL print — 1 is truthy
This WILL print — non-empty string is truthy
```

Gotcha: unlike Python, an **empty list, dict, or set is truthy** in JTS GO. A condition like `if []` is true. To test whether a collection is empty, use `len()`: `if len(items) == 0`.

5.9 The Collection Types at a Glance

Lists, dicts, and sets are covered in depth in Chapter 11. This section gives the brief overview needed to read the rest of the book.

```
# A list is ordered and indexed from 0
lst = [10, 20, 30]
print(lst[0])          # 10
print(len(lst))        # 3
lst.append(40)         # add to the end
print(lst)             # [10, 20, 30, 40]

# A dict maps string keys to values
d = {"name": "JTS", "year": 2026}
print(d["name"])       # JTS
print(d.get("year"))   # 2026

# A set stores unique values
s = {1, 2, 2, 3}
print(s)               # {1, 2, 3}
print(2 in s)          # true
```

6. Variables and Scope

6.1 Creating Variables

Variables are created simply by assigning a value with a single `=`. No keyword such as `var`, `let`, or `int` is required, although `var` and `list` exist as keywords for specific contexts.

```
name = "Alice"          # a string variable
age = 25                # a number variable
price = 9.99            # a float variable
is_active = true        # a boolean variable
result = nil            # a nil variable
```

6.2 Reassignment

A variable can be assigned a new value at any time, and the new value may have a different type:

```
x = 10
print(type(x))      # number
x = "ten"
print(type(x))      # string
x = [1, 2]
print(type(x))      # list
```

6.3 Compound Assignment Operators

JTS GO supports the classic compound assignment operators. They update a variable in place:

```
a = 10
a += 5     # a = a + 5
print(a)   # 15
a -= 3     # a = a - 3
print(a)   # 12
a *= 2     # a = a * 2
print(a)   # 24
a /= 4     # a = a / 4
print(a)   # 6
```

The full set includes `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, and the bitwise forms `&=`, `|=`, `^=`, `<<=`, and `>>=` (Chapter 7).

6.4 Global Scope

Variables created at the top level of a program are global: they are visible everywhere after their definition, including inside functions.

```
greeting = "Hello"

func greet(name)
    print(greeting + ", " + name + "!")
end

greet("Alice")       # Hello, Alice!
```

6.5 Local Scope

Variables created inside a function are local to that function. They do not exist outside it, and they do not leak between calls. A local variable can share a name with a global variable; inside the function, the local wins.

```
x = 10                    # a global variable

func modify()
    x = 20                 # creates a NEW local variable x
    print("Inside: " + x)   # Inside: 20
end

modify()
print("Outside: " + x)      # Outside: 10
```

```
Inside: 20
Outside: 10
```

> **Important:** assigning to a name inside a function creates a **local** variable; it does not modify a global of the same name. This is a deliberate design choice that keeps functions predictable. (If you need shared mutable state, use a list or dict that both scopes can read and mutate.)

6.6 Variables Created Inside Blocks

Variables created inside an `if`, `while`, or `for` block behave like local variables of the enclosing scope. In particular, a loop variable assigned inside a loop is still visible after the loop:

```
for i in 0 to 5
    last = i
end
print(last)      # 4  (visible after the loop)
```

Implementation note: the JTS compiler performs a two-pass analysis of function bodies and *hoists* loop-assigned variables so that every local is declared before it is used. This guarantees that assignments inside loops — including implicitly-created loop counters — are always safe and correctly scoped. This fix (introduced in Version 2.1) eliminated a whole class of subtle bugs where variables assigned for the first time inside a loop could misbehave.

6.7 Deleting Variables with `del`

The `del` statement removes a variable, a list element, or a dict key:

```
x = 10
del x                       # x no longer exists

lst = [1, 2, 3, 4]
del lst[1]                  # remove the element at index 1
print(lst)                  # [1, 3, 4]

dct = {"a": 1, "b": 2, "c": 3}
del dct["b"]                # remove the key "b"
print(dct)                  # {c: 3, a: 1}
print(len(dct))             # 2
```

```
[1, 3, 4]
{c: 3, a: 1}
2
```

6.8 Constants by Convention

JTS GO has no built-in constant mechanism; all variables can be reassigned. By convention, programs written in the JTS community use uppercase names for values that are meant to stay fixed:

```
PI = 3.14159
APP_NAME = "JTS GO"
```

7. Operators

7.1 Arithmetic Operators

Operator	Meaning	Example	Result
`+`	Addition / concatenation	`10 + 3`	`13`
`-`	Subtraction	`10 - 3`	`7`
`*`	Multiplication	`10 * 3`	`30`
`/`	Division	`10 / 3`	`3.33333`
`%`	Modulo (remainder)	`10 % 3`	`1`
`//`	Floor division	`10 // 3`	`3`
`**`	Exponentiation	`2 ** 10`	`1024`
`-x`	Unary negation	`-5`	`-5`

Modulo with negative numbers

The modulo operator returns a result with the sign of the dividend. Some algorithms want a non-negative result; the `dates` scroll ships a `mod` helper for that purpose (Chapter 34).

```
print(10 % 3)      # 1
print(-10 % 3)     # -1
print(10 % -3)     # 1
```

7.2 Comparison Operators

Comparisons always produce a boolean (`true` or `false`):

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`true`
`!=`	Not equal to	`5 != 3`	`true`
`>`	Greater than	`5 > 3`	`true`
`<`	Less than	`5 < 3`	`false`
`>=`	Greater than or equal	`5 >= 5`	`true`
`<=`	Less than or equal	`5 <= 3`	`false`

Strings compare lexicographically (alphabetically):

```
print("apple" < "banana")    # true
print("a" < "A")             # false  (uppercase sorts first)
```

7.3 Logical Operators

Operator	Meaning	Example	Result
`and`	True if both sides are truthy	`true and false`	`false`
`or`	True if either side is truthy	`true or false`	`true`
`not`	Inverts a boolean	`not true`	`false`

```
age = 25
has_id = true

if age >= 18 and has_id
    print("Entry allowed")
end

if age < 13 or age > 65
    print("Discount applies")
end

is_raining = false
if not is_raining
    print("No umbrella needed")
end
```

7.4 Membership: `in`

The `in` operator tests membership. It works on strings (is a character present), lists, dicts (is a key present), and sets:

```
print("e" in "hello")        # true
print("z" in "hello")        # false
print(2 in [1, 2, 3])        # true
print("b" in {"a": 1, "b": 2})   # true  (key membership)
print(9 in {1, 2, 3})        # false
```

7.5 Identity: `is`

The `is` operator compares values for identity. For simple values (numbers, strings, booleans) it behaves like equality; it is also used with `nil`:

```
print(1 is 1)        # true
print(1 is 2)        # false
print("a" is "a")    # true
print(nil is nil)    # true
```

7.6 Bitwise Operators

JTS GO includes the full set of bitwise operators from C and Java. They operate on the binary representations of integers.

```
print(5 & 3)      # 1    (bitwise AND: 101 & 011 = 001)
print(5 | 3)      # 7    (bitwise OR:  101 | 011 = 111)
print(5 ^ 3)      # 6    (bitwise XOR: 101 ^ 011 = 110)
print(~5)         # -6   (bitwise NOT: two's complement)
print(1 << 4)     # 16   (left shift)
print(256 >> 2)  # 64   (right shift)
```

```
1
7
6
-6
16
64
```

Compound bitwise assignments update a variable in place:

```
a = 6
a |= 1       # 7
b = 7
b &= 3       # 3
c = 5
c ^= 3       # 6
d = 2
d <<= 3      # 16
e = 64
e >>= 2      # 16
print(a, b, c, d, e)   # [7, 3, 6, 16, 16]  (multi-argument print)
```

Why bitwise operators matter: the `numbers` scroll uses bitwise tests for `is_power_of_two` (a number is a power of two exactly when `n & (n - 1) == 0`), and bit shifts are the fastest way to multiply or divide by powers of two.

7.7 Operator Precedence

From highest to lowest binding:

Precedence	Operators
1 (highest)	`**`, unary `-`, `~`, `not`
2	`*` `/` `//` `%`
3	`+` `-`
4	`<<` `>>`
5	`&`
6	`^`
7	`|`
8	`<` `<=` `>` `>=`
9	`==` `!=` `is` `in`
10	`and`
11 (lowest)	`or`

When in doubt, use parentheses — they are cheap and make intent explicit:

```
total = (price + tax) * quantity
```

8. Control Flow

8.1 The `if` Family

Basic if

```
temperature = 75
if temperature > 80
    print("It's hot outside!")
end
```

Because the condition is false, nothing prints. The body of an `if` runs only when the condition is truthy (Chapter 5.8).

if / else

```
temperature = 75
if temperature > 80
    print("It's hot outside!")
else
    print("It's nice outside!")
end
```

```
It's nice outside!
```

Chained conditions with `else if` / `elif`

Both `else if` and `elif` are accepted. Conditions are evaluated top to bottom; the first true branch runs and the rest are skipped.

```
score = 85

if score >= 90
    print("Grade: A")
elif score >= 80
    print("Grade: B")
elif score >= 70
    print("Grade: C")
elif score >= 60
    print("Grade: D")
else
    print("Grade: F")
end
```

```
Grade: B
```

Nested if

```
age = 25
has_ticket = true

if age >= 18
    if has_ticket
        print("Entry allowed")
    else
        print("Need a ticket")
    end
else
    print("Must be 18 or older")
end
```

8.2 The `while` Loop

A `while` loop repeats its body as long as the condition is truthy. Make sure the condition eventually becomes false, or the loop never ends.

```
count = 0
while count < 5
    print(count)
    count = count + 1
end
```

```
0
1
2
3
4
```

Countdown

```
n = 5
while n > 0
    print(n)
    n = n - 1
end
print("Liftoff!")
```

```
5
4
3
2
1
Liftoff!
```

Infinite loops: the body below never ends — the condition `true` never becomes false. Do not run it without a way to interrupt your program.

```
# while true
#     print("stuck!")
# end
```

8.3 The `for` Loop

The `for` loop iterates over a numeric range. The syntax is:

```
for VARIABLE in START to END
    ...
end
```

- `START` is inclusive — the loop begins there.
- `END` is **exclusive** — the loop stops before it.
- To loop from 0 to `n` inclusive, write `for i in 0 to n + 1`.

```
for i in 0 to 5
    print(i)
end
```

```
0
1
2
3
4
```

Multiplication table

```
for i in 1 to 11
    print("5 x " + i + " = " + (5 * i))
end
```

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Summing a range

```
total = 0
for i in 1 to 101
    total = total + i
end
print("Sum of 1 to 100: " + total)
```

```
Sum of 1 to 100: 5050
```

Iterating over lists, dicts, and sets

The `for ... in` form also iterates over collections (Chapter 11).

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits
    print(fruit)
end
```

```
apple
banana
cherry
```

The `range()` function

The built-in `range()` produces a list of numbers and is a convenient alternative to `for ... to`:

```
print(range(5))        # [0, 1, 2, 3, 4]
print(range(2, 6))     # [2, 3, 4, 5]
for x in range(3)
    print("x = " + x)
end
```

8.4 `break` and `continue`

- `break` immediately exits the innermost loop.
- `continue` skips the rest of the current iteration and moves to the next one.

```
i = 0
while i < 10
    i = i + 1
    if i == 3
        continue          # skip printing 3
    end
    if i == 7
        break             # stop at 7
    end
    print(i)
end
```

```
1
2
4
5
6
```

8.5 Combining Control Flow

All constructs nest freely. This example finds even numbers and categorizes them:

```
for i in 1 to 21
    if i % 2 == 0
        if i <= 10
            print(i + " is a small even number")
        else
            print(i + " is a large even number")
        end
    end
end
```

8.6 `assert`

`assert` checks that a condition is true; if it is false, the program stops with an error. An optional message can follow the condition after a comma.

```
assert 1 + 1 == 2
assert 5 > 3, "five should be bigger than three"
print("asserts ok")
```

```
asserts ok
```

Asserts are ideal for validating assumptions and for lightweight testing (Chapter 42).

8.7 Summary of Statement Forms

Construct	Syntax	Terminator
If	`if condition`	`end`
Else if	`elif condition`	`end`
Else	`else`	`end`
While	`while condition`	`end`
For over range	`for i in start to end`	`end`
For over collection	`for x in collection`	`end`
Delete	`del name`	—
Assert	`assert condition, message`	—

9. Functions

9.1 Defining a Function

Functions are defined with the `func` keyword, a name, optional parameters, a body, and `end`.

```
func greet()
    print("Hello from JTS GO!")
end

greet()    # Hello from JTS GO!
```

9.2 Parameters

Parameters are local variables that receive values at call time. Separate multiple parameters with commas.

```
func say_hello(name)
    print("Hello, " + name + "!")
end

func add(a, b)
    return a + b
end

say_hello("Alice")          # Hello, Alice!
result = add(3, 4)
print("3 + 4 = " + result)  # 3 + 4 = 7
```

9.3 Return Values

Use `return` to send a value back. A function can return different values depending on its logic:

```
func absolute(n)
    if n < 0
        return -n
    else
        return n
    end
end

print(absolute(-7))    # 7
print(absolute(7))     # 7
```

If a function reaches the end without a `return`, it returns `nil`:

```
func say_hi()
    print("Hi!")
end

result = say_hi()
print(result)    # nil  (the "Hi!" was printed as a side effect)
```

9.4 Functions Calling Functions

Complex behavior is built from small functions that call one another:

```
func square(x)
    return x * x
end

func sum_of_squares(a, b)
    return square(a) + square(b)
end

print("3^2 + 4^2 = " + sum_of_squares(3, 4))    # 3^2 + 4^2 = 25
```

9.5 Recursion

A function may call itself. Recursion is the natural way to express problems that decompose into smaller versions of themselves.

Factorial

```
func factorial(n)
    if n <= 1
        return 1
    else
        return n * factorial(n - 1)
    end
end

print(factorial(5))     # 120
print(factorial(10))    # 3628800
```

Trace how `factorial(5)` unwinds:

```
factorial(5) = 5 * factorial(4)
             = 5 * 4 * factorial(3)
             = 5 * 4 * 3 * factorial(2)
             = 5 * 4 * 3 * 2 * factorial(1)
             = 5 * 4 * 3 * 2 * 1
             = 120
```

Fibonacci

```
func fibonacci(n)
    if n <= 1
        return n
    end
    return fibonacci(n - 1) + fibonacci(n - 2)
end

for i in 0 to 12
    print("fib(" + i + ") = " + fibonacci(i))
end
```

```
fib(0) = 0
fib(1) = 1
fib(2) = 1
fib(3) = 2
fib(4) = 3
fib(5) = 5
fib(6) = 8
fib(7) = 13
fib(8) = 21
fib(9) = 34
fib(10) = 55
fib(11) = 89
```

Countdown

```
func countdown(n)
    if n <= 0
        print("Go!")
    else
        print(n)
        countdown(n - 1)
    end
end

countdown(5)
```

```
5
4
3
2
1
Go!
```

9.6 Scope Rules (Recap)

Variables created inside a function are local. Variables created at the top level are global and readable from anywhere. Assigning a name inside a function creates a local (Chapter 6).

```
greeting = "Hello"              # global

func greet(name)
    local_message = "Hi"        # local — invisible outside
    print(greeting + ", " + name + "!")
end

greet("Alice")                   # Hello, Alice!
# print(local_message)          # Error — no such variable here
```

9.7 Closures

A function defined *inside* another function can capture the surrounding function's variables and parameters. Such a function is called a closure, and it remembers its environment even after the outer function has returned.

```
func make_adder(x)
    func add(y)
        return x + y
    end
    return add
end

add5 = make_adder(5)
print(add5(10))      # 15
print(add5(100))     # 105
```

Each call to the factory creates an independent closure with its own captured state. State can also be shared through a mutable object such as a list:

```
func make_counter()
    state = []
    func increment()
        state.append(len(state))
        return len(state)
    end
    return increment
end

c1 = make_counter()
print(c1())    # 1
print(c1())    # 2
print(c1())    # 3
c2 = make_counter()
print(c2())    # 1   (independent counter)
print(c1())    # 4   (c1 still remembers its own state)
```

Closures nest to any depth and can capture parameters:

```
func multiplier(factor)
    func mul(n)
        return factor * n
    end
    return mul
end

double = multiplier(2)
triple_m = multiplier(3)
print(double(21))      # 42
print(triple_m(14))    # 42
```

9.8 Generators with `yield`

A function that contains `yield` becomes a generator. Calling it returns a generator object instead of running the body. Each call to the built-in `next()` runs the body up to the next `yield` and returns the yielded value, suspending execution until the next call.

```
func naturals(n)
    i = 0
    while i < n
        yield i
        i = i + 1
    end
end

g = naturals(3)
print(next(g))    # 0
print(next(g))    # 1
print(next(g))    # 2
```

```
0
1
2
```

Generators are ideal for lazily producing sequences that would be wasteful to materialize all at once.

9.9 Practical Function Patterns

Even / odd helpers

```
func is_even(n)
    return n % 2 == 0
end

func is_odd(n)
    return not is_even(n)
end

for i in 0 to 10
    if is_even(i)
        print(i + " is even")
    else
        print(i + " is odd")
    end
end
```

Power by repeated multiplication

```
func power(base, exp)
    result = 1
    i = 0
    while i < exp
        result = result * base
        i = i + 1
    end
    return result
end

print(power(2, 10))    # 1024
print(power(5, 3))     # 125
```

9.10 Summary

Feature	Syntax
Define	`func name(params) ... end`

Return	`return expression`
Call	`name(arguments)`
Recursion	A function calls itself
Closure	An inner function captures the enclosing scope
Generator	`yield value` inside a function; drive with `next(g)`

10. Object-Oriented Programming

10.1 Classes and Instances

JTS GO supports classes, methods, and inheritance. A class is a blueprint; an instance is an object created from it. The constructor is the `init` method, and instances are created with `new`.

```
class Animal
    func init(self, name)
        self.name = name
    end
end

a = new Animal("Cat")
print(a.name)    # Cat
```

10.2 The `self` Keyword

Every method takes `self` as its first parameter. Inside a method, `self` refers to the object the method was called on. Fields are simply attributes stored on `self` with `self.field = value`.

```
class Animal
    func init(self, name)
        self.name = name
    end

    func speak(self)
        print(self.name + " says hello!")
    end

    func greet(self, other)
        print(self.name + " greets " + other + "!")
    end
end

a = new Animal("Cat")
a.speak()          # Cat says hello!
a.greet("Dog")     # Cat greets Dog!
print(a.name)      # Cat
```

10.3 Inheritance with `extends`

A class can inherit from another with `extends`. The subclass receives all methods and fields of the parent, and can add its own. Instances are created with `new` using the subclass name.

```
class Animal
    func init(self, name)
        self.name = name
    end

    func speak(self)
        print(self.name + " makes a sound")
    end
end

class Dog extends Animal
    func bark(self)
        print(self.name + " barks!")
    end
end

d = new Dog("Rex")
d.speak()          # Rex makes a sound   (inherited method)
d.bark()           # Rex barks!          (own method)
print(d.name)      # Rex
```

Note: the `super` keyword exists in the language for advanced cases, and the `extends` keyword declares the parent class. Chapter 10.5 shows a complete example.

10.4 Fields Are Open

Objects are flexible: you can add or read fields on an instance at any time.

```
class Point
    func init(self, x, y)
        self.x = x
        self.y = y
    end
end

p = new Point(3, 4)
print(p.x)        # 3
p.label = "origin-relative"    # add a brand-new field
print(p.label)
```

10.5 A Complete OOP Example

This example models a bank account with inheritance and method calls:

```
class Account
    func init(self, owner)
        self.owner = owner
        self.balance = 0
    end

    func deposit(self, amount)
        self.balance = self.balance + amount
    end

    func withdraw(self, amount)
        if amount <= self.balance
            self.balance = self.balance - amount
        else
            print("Insufficient funds")
        end
    end

    func report(self)
        print(self.owner + " has $" + self.balance)
    end
end

class SavingsAccount extends Account
    func add_interest(self, rate)
        self.balance = self.balance + self.balance * rate
    end
end

acc = new SavingsAccount("Alice")
acc.deposit(100)
acc.deposit(50)
acc.withdraw(30)
acc.add_interest(0.05)
acc.report()
```

```
Alice has $126
```

10.6 When to Use Classes

- When data and behavior travel together (a `Dog` with a `name` and a `bark()`).
- When you want multiple objects with the same shape but independent state.
- When inheritance expresses a natural hierarchy (`SavingsAccount` is an `Account`).

For simple groupings of data, a dict (Chapter 11) is often enough. Prefer the simplest tool that fits.

11. Collections: Lists, Dicts, and Sets

11.1 Lists

A list is an ordered collection of values. Lists are written with square brackets, hold any mixture of types, and support indexing from 0 (negative indices count from the end).

```
lst = [10, 20, 30]
print(lst[0])        # 10
print(lst[-1])       # 30
print(len(lst))      # 3

mixed = ["a", 42, true, nil, [1, 2]]
print(len(mixed))    # 5
```

Building lists

```
empty = []
squares = [1, 4, 9, 16]
repeated = [0] * 3   # ??? — see note below
```

> **Note:** list multiplication is not a core operator in JTS GO. To repeat a list, use the `seq` scroll's `repeat_times` (Chapter 29) or a loop.

Adding, removing, sorting

Methods are called directly on the list. The same operations also exist as global functions.

```python
lst = [3, 1, 2]

lst.append(4)       # add to the end          -> [3, 1, 2, 4]
lst.sort()          # sort ascending          -> [1, 2, 3, 4]
print(lst)          # [1, 2, 3, 4]

lst.remove(2)       # remove first occurrence of the value
print(lst)          # [1, 3, 4]

lst.pop()           # remove and return the last element
print(lst)          # [1, 3]

lst.insert(1, 9)    # insert at an index
print(lst)          # [1, 9, 3]

lst.extend([7, 8])  # append all elements of another list
print(lst)          # [1, 9, 3, 7, 8]

lst.reverse()
print(lst)          # [8, 7, 3, 9, 1]
```

```
[1, 2, 3, 4]
[1, 3, 4]
[1, 3]
[1, 9, 3]
[1, 9, 3, 7, 8]
[8, 7, 3, 9, 1]
```

Global equivalents: `append(lst, x)`, `insert(lst, i, x)`, `extend(lst, other)`. Both styles work; pick one and be consistent.

Copying, clearing, finding

```python
original = [1, 2, 3]
copy = original.copy()     # an independent copy
print(copy)                # [1, 2, 3]

print([1, 2, 1, 2].index(2))   # 1   (first index of value 2)
print([1, 2, 1, 2].count(1))   # 2   (how many times 1 appears)
```

Note: `list` is a keyword (it denotes the list type in certain contexts), so avoid naming a variable `list`. Use `lst`, `items`, or `data`.

Iterating over a list

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits
    print(fruit)
end
```

Comprehensions

List comprehensions build a list from an expression, an iteration, and an optional filter:

```
squares = [x * x for x in range(1, 6)]
print(squares)                 # [1, 4, 9, 16, 25]

evens = [x for x in range(10) if x % 2 == 0]
print(evens)                   # [0, 2, 4, 6, 8]

names = ["alice", "bob", "carol"]
print([n.upper() for n in names])    # [ALICE, BOB, CAROL]

neg = [-3, -1, 0, 2, 4]
print([x for x in neg if x > 0])     # [2, 4]

nested = [[a for a in range(b)] for b in range(1, 4)]
print(nested)                        # [[0], [0, 1], [0, 1, 2]]
```

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
[ALICE, BOB, CAROL]
[2, 4]
[[0], [0, 1], [0, 1, 2]]
```

11.2 Dicts

A dict (dictionary) maps string keys to values. Literal keys can be written with quotes (`{"name": "JTS"}`) or without (`{name: "JTS"}`).

```
d = {"name": "JTS", "year": 2026, "active": true}
print(d["name"])      # JTS
print(d["year"])      # 2026
```

Reading keys safely

```
d = {"a": 1, "b": 2}
print(d.get("a"))           # 1
print(d.get("missing"))     # nil
print(d.get("missing", 99)) # 99   (a default value)
print(d.has("b"))           # true
print(d.has("z"))           # false
```

Adding and updating keys

Add or merge keys with `update`. Unlike lists, dicts cannot be modified with subscript assignment (`d["k"] = v` is an error); use `update` instead.

```
d = {"a": 1, "b": 2}
d.update({"c": 3})          # method form
print(d.has("c"))           # true
update(d, {"z": 26})        # global form
print(d.get("z"))           # 26
print(d)                    # {c: 3, a: 1, b: 2, z: 26}
```

Gotcha: a printed dict does **not** show keys in insertion order — the display order is not guaranteed. Never rely on dict ordering. (Note also that `del d["b"]` from Chapter 6 works for deleting keys.)

Viewing contents

```
d = {"a": 1, "b": 2}
print(d.keys())      # [a, b]
print(d.values())    # [1, 2]
print(d.items())     # [[a, 1], [b, 2]]

for k in d.keys()
    print(k + " = " + d[k])
end
```

Membership and deletion

```
d = {"a": 1, "b": 2}
print("a" in d)      # true   (key membership)
del d["a"]
print(d.has("a"))    # false
```

Iterating a dict's keys

```
d = {"name": "JTS", "year": 2026}
for key in d.keys()
    print(key)
end
```

11.3 Sets

A set is an unordered collection of unique values. Set literals use braces with two or more elements. Use the `set` function to build a set from a list or string, or to create a single-element set.

```
s = {1, 2, 3, 2, 1}       # duplicates are discarded
print(s)                   # {1, 2, 3}
print(len(s))              # 3

t = set([4, 5, 6])
print(t)                   # {4, 5, 6}

single = set([1])
print(single)              # {1}

chars = set("abc")
print(chars)               # {a, b, c}
```

Set algebra operators

```
s = {1, 2, 3}
t = set([4, 5, 6])

print(s | t)                       # union        -> {1, 2, 3, 4, 5, 6}
print(s & set([2, 3, 4]))          # intersection -> {2, 3}
print(s - set([1]))                # difference   -> {2, 3}
print(s ^ set([2, 7]))             # symmetric difference -> {1, 3, 7}
```

```
{1, 2, 3, 4, 5, 6}
{2, 3}
{2, 3}
{1, 3, 7}
```

Set functions

```
s = {1, 2, 3}
set_add(s, 7)
print(7 in s)            # true
print(set_contains(s, 7)) # true
set_remove(s, 3)
print(3 in s)            # false
print(len(s))            # 3
```

Iterating a set

```
for item in {1, 2, 3}
    print("item: " + item)
end
```

11.4 Choosing the Right Collection

Need	Use
Ordered values, indexed by position	List
Look up values by string key	Dict
Unique values, fast membership tests, set algebra	Set
Records / structured data that travels together	Dict, or a class instance

12. Strings in Depth

12.1 Creating Strings

```
greeting = "Hello, World!"
empty = ""
```

12.2 Length, Indexing, and Slicing

```
s = "Hello"
print(len(s))        # 5
print(s[0])          # H
print(s[4])          # o
print(s[-1])         # o

print(s.substring(1, 3))     # el     (start inclusive, end exclusive)
print(s.substring(0, 5))     # Hello
print(s.substring(2))        # llo    (from index 2 to the end)
```

```
5
H
o
o
el
Hello
llo
```

`substring(start, end)` treats `end` as **exclusive** — exactly like Java's `substring`. With one argument, it runs from `start` to the end of the string.

> **Gotcha:** because the end index is exclusive, "all but the last character" is `s.substring(0, len(s) - 1)` and "from index 1 onward" is `s.substring(1)`.

12.3 Case Conversion and Inspection

```
s = "Hello World"
print(s.upper())        # HELLO WORLD
print(s.lower())        # hello world
print(s.capitalize())   # Hello world      (first letter only)
print(s.title())        # Hello World      (every word capitalized)
print(s.swapcase())     # hELLO wORLD
```

Predicates

```
print("123".is_digit())    # true
print("abc".is_alpha())    # true
print("a1".is_alnum())     # true
print(" ".is_space())      # true
print("ABC".is_upper())    # true
print("abc".is_lower())    # true
```

12.4 Trimming and Padding

```
print("  hi  ".trim())     # hi
print("  x  ".lstrip())    # "x  "
print("  x  ".rstrip())    # "  x"

print("7".zfill(3))        # 007
print("hi".ljust(5))       # "hi   "
print("hi".rjust(5))       # "   hi"
print("hi".center(6))      # "  hi  "
```

12.5 Searching and Substitution

```
s = "Hello World"
print(s.contains("World"))       # true
print(s.contains("world"))       # false   (case-sensitive)
print(s.starts_with("Hello"))    # true
print(s.ends_with("World"))      # true
print(s.find("o"))               # 4       (first index of the substring)
print(s.replace("World", "JTS")) # Hello JTS
```

12.6 Splitting and Joining

`split` divides a string on a delimiter and returns a list. `join` assembles a list into a string; it expects the **separator first**, either as a global call or as a method on the separator string.

```
print("a,b,c".split(","))        # [a, b, c]
print("a\nb\nc".splitlines())    # [a, b, c]
print(join("-", ["a", "b", "c"]))  # a-b-c
print("-".join(["a", "b", "c"]))   # a-b-c   (equivalent)
```

12.7 Counting and Formatting

```
s = "hello"
print(s.count("l"))      # 2
print(s.find("l"))       # 2

name = "JTS"
print("name={0},age={1}".format(name, 30))   # name=JTS,age=30
```

12.8 Iterating Characters

```
word = "hello"
for c in word
    print(c)
end
```

12.9 String Utilities in the Standard Library

For heavier string work — slugs, casing conversions, ciphers, word counts — the `text` and `strings` scrolls provide ready-made functions. They are documented in Chapters 26 and 27.

13. Error Handling

13.1 `throw` and `catch`

JTS GO uses an exception model with `try`, `catch`, and `throw`. You can throw any value (usually a descriptive string), and a `catch` clause receives it.

```
try
    throw "Something went wrong!"
catch e
    print("Caught: " + e)
end
```

```
Caught: Something went wrong!
```

13.2 `finally`

A `finally` block runs whether or not an exception was thrown. It is the right place for cleanup that must always happen — closing files, resetting state, and so on.

```
try
    throw "failure"
catch e
    print("caught " + e)
finally
    print("cleanup always runs")
end
```

```
caught failure
cleanup always runs
```

13.3 Catching Only What You Need

A common pattern is to attempt an operation and fall back on a sensible default when it fails:

```
func safe_read(path)
    try
        return read_file(path)
    catch e
        return ""
    end
end

content = safe_read("maybe_missing.txt")
print("content is: '" + content + "'")
```

13.4 Runtime Errors

When the virtual machine hits an invalid operation — calling a non-function, indexing out of bounds, passing the wrong number of arguments — it reports a runtime error with the source line, for example:

```
JTS GO: Can only call functions.
[line 12] in script
Runtime error.
```

These messages name the operation and the line number, which points you straight to the offending statement. A `try/catch` cannot always intercept internal VM errors, so the first line of defence is correct code; the second is `assert` and careful validation.

13.5 Defensive Programming Checklist

- Validate inputs before using them (`type()`, `len()`, `has()`).
- Use `d.get(key, default)` instead of assuming a key exists.
- Use `try/catch` around operations that can legitimately fail (file I/O).
- Use `assert` to document invariants that must always hold.

14. Core Built-in Functions

JTS GO ships with a set of built-in functions that are always available — no imports required. This chapter documents the core ones. The complete function table is listed in Appendix C.

14.1 `print(value, ...)`

Writes a value to the console followed by a newline. `print` accepts any value — strings, numbers, booleans, `nil`, lists, dicts, sets, and instances — and prints its natural representation.

```
print("Hello, World!")
print(42)
print(3.14)
print(true)
print(nil)
print([1, 2, 3])
print(10 + 5)          # prints the result of an expression: 15
print(len("JTS"))      # prints the result of a function call: 3
```

```
Hello, World!
42
3.14
true
nil
[1, 2, 3]
15
3
```

When given several arguments, `print` displays them as a list:

```
print("a", "b", "c")    # [a, b, c]
```

> **Tip:** to build formatted messages, use string concatenation instead of multiple arguments: `print("Age: " + age)` prints `Age: 25` rather than a list.

14.2 `input(prompt?)`

Reads a line of text from the user. The optional prompt is displayed first. The return value is always a string.

```
name = input("What is your name? ")
print("Hello, " + name + "!")
```

Because the result is a string, numeric input must be converted before arithmetic (Chapter 14.6).

14.3 `len(value)`

Returns the length of a string (number of characters), a list (number of elements), a dict (number of keys), or a set (number of elements).

```
print(len("JTS"))        # 3
print(len([1, 2, 3]))    # 3
print(len({"a": 1}))     # 1
print(len({1, 2, 3}))    # 3
```

14.4 `type(value)`

Returns the type of a value as a string. The possible results are `number`, `string`, `boolean`, `nil`, `list`, `dict`, `set`, `function`, and the class name of instances.

```
print(type(42))           # number
print(type("hi"))         # string
print(type(true))         # boolean
print(type(nil))          # nil
print(type([1]))          # list
print(type({"a": 1}))     # dict
print(type({1, 2}))       # set
print(type(func f() end)) # function  (a function literal)
```

Comparing the result to a literal is the standard way to branch on type:

```
if type(value) == "string"
    print("It's a string!")
end
```

14.5 `str(value)` and `number(value)`

`str()` converts a value to its string representation. `number()` attempts to convert a string to a number.

```
print(str(123))           # 123
print(str(true))          # true
print(number("42"))       # 42
print(number("3.5"))      # 3.5
```

`str()` is what `print` uses internally, and it is also useful for building lists of text from numbers:

```
out = []
for i in 0 to 4
    out.append(str(i))
end
print(out)                # [0, 1, 2, 3, 4]
```

14.6 `int(value)`, `float(value)`, `bool(value)`

These conversions are precise tools:

- `int(x)` converts to an integer, truncating any fractional part.
- `float(x)` converts to a floating-point number.
- `bool(x)` converts a value to a boolean following truthiness rules (Chapter 5.8).

```
print(int("42"))        # 42
print(int(3.99))        # 3    (truncates toward zero)
print(int("3.7"))       # 3
print(float("3.5"))     # 3.5
print(float(2))         # 2
print(bool("x"))        # true
print(bool(""))         # false
print(bool(0))          # false
print(bool(1))          # true
```

Reading a number from the user, then, looks like:

```
age = input("How old are you? ")
age_num = number(age)
if age_num >= 18
    print("Adult")
else
    print("Minor")
end
```

14.7 `append(list, value)` and `length(value)`

`append` adds a value to the end of a list. It can be called as a global function or as a method on the list. `length` is a global synonym for the length of a string or collection.

```
lst = []
append(lst, "first")
lst.append("second")
print(lst)              # [first, second]
print(length("hello"))  # 5
```

14.8 `format(template, args...)`

`format` substitutes numbered placeholders `{0}`, `{1}`, ... with arguments:

```
name = "JTS"
version = "2.1"
print(format("{} is v{}", name, version))   # JTS is v2.1
print(format("name={0},age={1}", name, 30)) # name=JTS,age=30
```

It can also be invoked as a method on the template string:

```
print("{} + {} = {}".format(1, 2, 3))       # 1 + 2 = 3
```

15. Math Built-in Functions

15.1 Arithmetic that Operates by Name: `math(op, x)`

The generic `math` function applies a named mathematical operation to a number. Valid operation names are `"sin"`, `"cos"`, `"tan"`, `"sqrt"`, `"abs"`, `"log"`, `"exp"`, `"pow"`, `"floor"`, `"ceil"`, and `"round"`.

```
print(math("sqrt", 16))    # 4
print(math("abs", -9))     # 9
print(math("floor", 3.9))  # 3
print(math("ceil", 3.1))   # 4
```

Most of these operations also have dedicated functions, which are usually clearer to read.

15.2 Dedicated Math Functions

Function	Returns	Example	Result
`sqrt(x)`	Square root	`sqrt(16)`	`4`
`sin(x)`	Sine (radians)	`sin(0)`	`0`
`cos(x)`	Cosine (radians)	`cos(0)`	`1`
`tan(x)`	Tangent (radians)	`tan(0)`	`0`
`log(x)`	Natural logarithm	`log(1)`	`0`
`exp(x)`	`e` raised to x	`exp(0)`	`1`
`abs(x)`	Absolute value	`abs(-5)`	`5`
`pow(a, b)`	`a` to the power `b`	`pow(2, 10)`	`1024`
`round(x, d?)`	Rounds x; optional decimal places	`round(3.14159, 2)`	`3.14`
`floor(x)`	Rounds down	`floor(3.9)`	`3`
`ceil(x)`	Rounds up	`ceil(3.1)`	`4`

```
print(sqrt(16))          # 4
print(abs(-5))           # 5
print(min(3, 1, 2))      # 1
print(max(3, 1, 2))      # 3
print(sum([1, 2, 3]))    # 6
print(pow(2, 10))        # 1024
print(round(3.14159))    # 3
print(round(3.14159, 2)) # 3.14
print(floor(3.9))        # 3
print(ceil(3.1))         # 4
print(sin(0))            # 0
print(cos(0))            # 1
print(log(1))            # 0
print(exp(0))            # 1
```

```
4
5
1
3
6
1024
3
3.14
3
4
0
1
0
1
```

15.3 `min(...)`, `max(...)`, `sum(list)`

- `min(a, b, c, ...)` — smallest of the arguments.
- `max(a, b, c, ...)` — largest of the arguments.
- `sum(list)` — total of all elements in a list.

```
print(min(5, 2, 9))      # 2
print(max(5, 2, 9))      # 9
print(sum([1, 2, 3, 4])) # 10
```

These three are the building blocks of the statistics scroll (Chapter 28).

15.4 `range(start?, end?, step?)`

`range` builds a list of numbers. With one argument it produces `0 ... n-1`; with two it runs from `start` to `end - 1`.

```
print(range(5))          # [0, 1, 2, 3, 4]
print(range(2, 6))       # [2, 3, 4, 5]
for i in range(3)
    print(i)             # 0, 1, 2
end
```

15.5 Random Numbers

The random family is documented in Chapter 19.

16. String Built-in Functions

JTS GO's string functions are called as methods on string values. They are pure — they never modify the string itself; they return a new string.

16.1 Case Conversion

```
s = "Hello World"
print(s.upper())        # HELLO WORLD
print(s.lower())        # hello world
print(s.capitalize())   # Hello world    (first character only)
print(s.title())        # Hello World    (capitalizes every word)
print(s.swapcase())     # hELLO wORLD
```

16.2 Trimming and Padding

```
print("  hi  ".trim())     # hi
print("  x  ".lstrip())    # "x  "     (left only)
print("  x  ".rstrip())    # "  x"     (right only)

print("7".zfill(3))       # 007
print("hi".ljust(5))      # "hi   "   (pad right)
print("hi".rjust(5))      # "   hi"   (pad left)
print("hi".center(6))     # "  hi  "  (pad both sides)
```

16.3 Searching

```
s = "Hello World"
print(s.contains("World"))      # true
print(s.starts_with("Hello"))   # true
print(s.ends_with("World"))     # true
print(s.find("o"))              # 4    (first index of the substring)
print(s.count("l"))             # 3    (occurrences)
```

All of these are case-sensitive.

16.4 Character Tests

```
print("123".is_digit())    # true
print("abc".is_alpha())    # true
print("a1".is_alnum())     # true
print(" ".is_space())      # true
print("ABC".is_upper())    # true
print("abc".is_lower())    # true
```

These predicates are used heavily by the `text` scroll to classify characters while scanning.

16.5 Splitting

```
print("a,b,c".split(","))        # [a, b, c]
print("a b c".split(" "))        # [a, b, c]
print("a\nb\nc".splitlines())    # [a, b, c]
```

Gotcha: `splitlines()` keeps a trailing empty element for text that ends with a newline. The `fs` scroll's `read_lines` strips this automatically (Chapter 33).

16.6 Joining

`join` combines a list into a string with a separator between elements. The separator comes **first**, either as the argument of a global call or as the receiver of a method call.

```
print(join("-", ["a", "b", "c"]))   # a-b-c
print("-".join(["a", "b", "c"]))    # a-b-c
```

16.7 Extracting Substrings

```
s = "Hello"
print(s.substring(1, 3))     # el     (start inclusive, end exclusive)
print(s.substring(1))        # ello   (to the end)
print(s.replace("Hello", "Hi"))    # Hi
```

16.8 `format` on Strings

```
print("{} and {}".format(1, 2))        # 1 and 2
print("name={0},age={1}".format("A", 30))  # name=A,age=30
```

17. List Built-in Functions

List functions are available both as methods on a list and as global functions taking the list first. Methods modify the list in place (they are not pure).

17.1 Growth: `append`, `insert`, `extend`

```
lst = [1, 2, 3]
lst.append(4)               # add to the end
print(lst)                  # [1, 2, 3, 4]

lst.insert(1, 9)            # insert 9 at index 1
print(lst)                  # [1, 9, 2, 3, 4]

lst.extend([7, 8])          # append every element of the other list
print(lst)                  # [1, 9, 2, 3, 4, 7, 8]
```

Global forms: `append(lst, 4)`, `insert(lst, 1, 9)`, `extend(lst, [7, 8])`.

17.2 Shrinkage: `remove`, `pop`, `clear`

```
lst = [1, 2, 3, 4]
lst.remove(2)               # remove the first occurrence of the value 2
print(lst)                  # [1, 3, 4]

last = lst.pop()            # remove and return the last element
print(last)                 # 4
print(lst)                  # [1, 3]

lst.clear()                 # remove everything
print(lst)                  # []
```

17.3 Ordering: `sort`, `reverse`

```
lst = [3, 1, 2]
lst.sort()
print(lst)                  # [1, 2, 3]

lst.reverse()
print(lst)                  # [3, 2, 1]
```

17.4 Information: `index`, `count`, `len`

```
print([1, 2, 1, 2].index(2))    # 1   (first index of value 2)
print([1, 2, 1, 2].count(1))    # 2   (how many times 1 appears)
print(len([1, 2, 3]))           # 3
```

17.5 Copies: `copy`

```
original = [1, 2, 3]
shallow = original.copy()      # an independent list
shallow.append(4)
print(original)                # [1, 2, 3]  (unchanged)
print(shallow)                 # [1, 2, 3, 4]
```

Note: `copy()` copies the list itself. Elements that are themselves lists are shared, not copied recursively. For a fully independent copy, build one element at a time.

17.6 Indexing and Deletion

```
lst = [10, 20, 30]
print(lst[0])        # 10
print(lst[-1])       # 30
lst[0] = 99          # replace an element
print(lst)           # [99, 20, 30]
del lst[1]           # delete an element
print(lst)           # [99, 30]
```

Unlike dicts (Chapter 11), lists *can* be modified with subscript assignment.

18. Dict and Set Built-in Functions

18.1 Dict Methods

Dict methods are called on the dict value. They take the dict as an implicit receiver.

`d.get(key, default?)`

Returns the value for a key, or a default (by default `nil`) when the key is missing.

```
d = {"a": 1, "b": 2}
print(d.get("a"))            # 1
print(d.get("missing"))      # nil
print(d.get("missing", 99))  # 99
```

`d.has(key)`

```
d = {"a": 1}
print(d.has("a"))    # true
print(d.has("z"))    # false
```

`d.update(other)`

Merges another dict into `d`. This is the way to add or replace keys (subscript assignment is not supported for dicts).

```
d = {"a": 1}
d.update({"b": 2, "c": 3})
print(d.has("c"))    # true
print(d.get("b"))    # 2
```

`d.keys()`, `d.values()`, `d.items()`

```
d = {"a": 1, "b": 2}
print(d.keys())       # [a, b]
print(d.values())     # [1, 2]
print(d.items())      # [[a, 1], [b, 2]]
```

`items()` returns a list of `[key, value]` pairs — perfect for iteration:

```
d = {"x": 10, "y": 20}
for pair in d.items()
    print(pair[0] + " -> " + pair[1])
end
```

18.2 Set Functions

Sets can be built with `set(...)`, manipulated with the `set_*` functions, and combined with the operators `|`, `&`, `-`, and `^`.

Creating sets

```
s1 = {1, 2, 3}          # set literal (two or more elements)
s2 = set([4, 5, 6])     # set from a list
s3 = set("abc")         # set from a string -> {a, b, c}
s4 = set([1])           # single-element set (a one-element brace is invalid)
```

Membership and editing

```
s = {1, 2, 3}
print(2 in s)               # true
print(set_contains(s, 2))   # true

set_add(s, 7)               # adds 7 (a no-op if already present)
print(7 in s)               # true

set_remove(s, 3)            # removes 3
print(3 in s)               # false

print(len(s))               # 3
```

Set algebra

```
a = {1, 2, 3}
b = set([2, 3, 4])

print(set_union(a, b))          # {1, 2, 3, 4}
print(set_intersection(a, b))   # {2, 3}
print(set_difference(a, b))     # {1}
```

Each `set_*` function has an operator equivalent: `a | b`, `a & b`, `a - b`, `a ^ b`.

19. Random Numbers

19.1 `rand()`

Returns a random number in the interval `[0, 1)` (greater than or equal to 0, less than 1).

```
print(rand() >= 0 and rand() < 1)   # true
print(rand())                        # some value in [0, 1)
```

19.2 `randint(a, b)`

Returns a random integer between `a` and `b`, **inclusive** on both ends.

```
print(randint(1, 6))    # a random die roll: 1, 2, 3, 4, 5, or 6
```

19.3 `seed(n)`

Seeds the random generator. Using the same seed reproduces the same sequence — invaluable for testing and for reproducible machine-learning runs.

```
seed(42)
print(rand())
print(rand())

seed(42)                # reseed
print(rand())           # identical to the first value above
print(rand())           # identical to the second value above
```

19.4 `shuffle(list)`

Randomly reorders a list **in place**. Note that `shuffle` is a positional function — call it as `shuffle(lst)`, not `lst.shuffle()`.

```
lst = [1, 2, 3, 4, 5]
shuffle(lst)
print(lst)    # some random ordering of 1..5
```

> **Gotcha:** because `shuffle` modifies its argument, use `lst.copy()` first if you need to keep the original order — exactly what the `seq` scroll does in `shuffle_copy` (Chapter 29).

19.5 A Dice Roller

```
func roll_dice(sides, times)
    out = []
    for i in 0 to times
        out.append(randint(1, sides))
    end
    return out
end

print(roll_dice(6, 5))    # e.g. [4, 1, 6, 3, 5]
```

20. File and OS Functions

20.1 `read_file(path)` and `write_file(path, content)`

`read_file` returns the entire contents of a text file as a string. `write_file` writes a string to a file, replacing whatever was there.

```
write_file("notes.txt", "Hello from JTS GO!")
content = read_file("notes.txt")
print(content)                    # Hello from JTS GO!
```

20.2 `file_exists(path)`

```
print(file_exists("notes.txt"))    # true
print(file_exists("nope.txt"))     # false
```

20.3 `import_file(path)`

Loads and executes another `.jts` file at the current position. This is the low-level mechanism the scroll system is built on (Chapter 25).

```
import_file("helpers.jts")    # runs helpers.jts, making its globals available
```

20.4 Environment and Process

```
print(cwd())                  # the current working directory
print(len(env("PATH")))       # the length of the PATH environment variable
print(args())                 # a list of command-line arguments
exit(0)                       # terminate the program with exit code 0
```

- `env(name)` — value of an environment variable.
- `cwd()` — current working directory.
- `args()` — command-line arguments as a list.
- `exit(code?)` — terminate the program immediately.

20.5 Reading a Whole File, Line by Line

```
write_file("data.txt", "alpha\nbeta\ngamma\n")
lines = read_file("data.txt").splitlines()
for line in lines
    if line == ""
        continue
    end
    print("line: " + line)
end
```

For convenience, the `fs` scroll wraps all of this (Chapter 33).

21. JSON

21.1 `json_parse(text)`

Parses a JSON string into JTS GO values: JSON objects become dicts, arrays become lists, and numbers, strings, booleans, and `null` map to their natural equivalents.

```
parsed = json_parse('{"name": "JTS", "year": 2026, "tags": ["a", "b"], "ok": true}')
print(parsed["name"])       # JTS
print(parsed["year"])       # 2026
print(parsed["tags"])       # [a, b]
print(parsed["ok"])         # true
```

21.2 `json_stringify(value)`

Converts a JTS GO value into a JSON string. Dicts, lists, numbers, strings, booleans, and `nil` are all supported.

```
print(json_stringify({"a": 1, "b": [2, 3]}))    # {"a":1,"b":[2,3]}
print(json_stringify([1, "two", false, nil]))   # [1,"two",false,null]
```

21.3 Round Trip

```
config = {"name": "server", "port": 8080, "debug": true}
text = json_stringify(config)
print(text)

config_again = json_parse(text)
print(config_again["name"])    # server
print(config_again["port"])    # 8080
```

The `json` scroll wraps these functions for convenience (Chapter 36).

22. Time and Dates

22.1 `now()`

Returns the current time as Unix epoch seconds — the number of seconds since 1970-01-01 00:00:00 UTC.

```
print(now())    # e.g. 1777892400
```

22.2 `sleep(seconds)`

Pauses the program for the given number of seconds. Fractions are allowed.

```
print("before")
sleep(1.0)              # wait 1 second
print("after")
```

22.3 `strftime(pattern, timestamp?)`

Formats a timestamp (default: the current time) according to a `strftime`-style pattern. This is the same pattern language used by the C standard library and Python's `strftime`.

```
print(strftime("%Y-%m-%d"))           # current date, e.g. 2026-08-04
print(strftime("%H:%M:%S"))           # current time, e.g. 14:30:00
print(strftime("%Y-%m-%d %H:%M:%S"))  # combined stamp
print(strftime("%A, %d %B %Y"))       # e.g. Tuesday, 04 August 2026
```

Common conversion specifiers:

Specifier	Meaning	Example
`%Y`	Year with century	`2026`
`%y`	Year without century	`26`
`%m`	Month (01–12)	`08`
`%d`	Day of month (01–31)	`04`
`%H`	Hour, 24-hour (00–23)	`14`
`%M`	Minute (00–59)	`30`
`%S`	Second (00–59)	`00`
`%a` / `%A`	Weekday, abbreviated / full	`Tue` / `Tuesday`

`%b` / `%B`	Month, abbreviated / full	`Aug` / `August`
`%j`	Day of year (001–366)	`216`
`%w`	Weekday as number (0 = Sunday)	`2`
`%s`	Unix epoch seconds	`1777892400`

> **Platform note:** `%u` (ISO weekday) and `%V` (ISO week) are not available in the Windows C runtime. The `dates` scroll computes ISO week numbers itself (Chapter 34).

22.4 Working with Epoch Arithmetic

Because timestamps are plain numbers, date math is straightforward: add 86,400 to move forward one day.

```
ts = now()
tomorrow = ts + 86400
print(strftime("%Y-%m-%d", ts))        # today
print(strftime("%Y-%m-%d", tomorrow))  # tomorrow
```

23. Web Development

23.1 Serving a Web Page: `http_server(port, html)` and `http_start(server)`

JTS GO can serve a complete website with two function calls. `http_server` creates a server for the given port and response body; `http_start` starts listening (blocking) and serves until interrupted.

```
html = "<h1>Hello from JTS GO!</h1><p>Served by the JTS web server.</p>"
srv = http_server(8080, html)
print("Open http://localhost:8080 in your browser")
http_start(srv)
```

If no response body is given, the server returns a default `JTS GO Server` page.

23.2 Fetching Web Content: `http_request(url)`

`http_request` performs a GET request and returns the response body as a string. Both `http://` and `https://` URLs are supported.

```
body = http_request("http://example.com")
print(len(body))    # length of the fetched page
```

23.3 A Complete Example: A Portfolio Page

Chapter 41 builds a full multi-section website in JTS GO. The key idea is simple: build the HTML as a string (concatenating parts is fine), then serve it.

```
part1 = "<html><head><title>JTS GO</title></head><body>"
part2 = "<h1>Welcome</h1><p>Built with JTS GO.</p>"
part3 = "</body></html>"

html = part1 + part2 + part3
srv = http_server(8080, html)
http_start(srv)
```

24. Machine Learning and Tensors

JTS GO embeds a small but genuine AI toolkit: matrices, tensor data, activation functions, and loss functions. This makes it possible to build and train neural networks without any external libraries (Chapter 40 trains a real network on the XOR problem).

24.1 `tensor(nested_list)`

Wraps nested list data into a tensor object used for numeric computation.

```
t = tensor([[1, 2, 3], [4, 5, 6]])
print(t)   # a tensor representation of the 2x3 data
```

24.2 `matrix(nested_list)`

Creates a matrix with the given rows. Matrices support matrix multiplication and are the workhorse of linear algebra.

```
m = matrix([[1, 2], [3, 4]])
print(m)
```

24.3 `matmul(a, b)`

Multiplies two matrices. The number of columns of `a` must equal the number of rows of `b`; otherwise an error is reported.

```
m1 = matrix([[1, 2], [3, 4]])
m2 = matrix([[5, 6], [7, 8]])
print(matmul(m1, m2))
```

```
[[19, 22], [43, 50]]
```

24.4 `sigmoid(x)`

The logistic function `1 / (1 + e^-x)`. It squashes any number into the range `(0, 1)` and is the classic activation function for output layers.

```
print(sigmoid(0))      # 0.5
print(sigmoid(3))      # 0.952574
print(sigmoid(-3))     # 0.0474259
```

24.5 `relu(x)`

The rectified linear unit: `max(0, x)`. It is the most widely used activation for hidden layers.

```
print(relu(2))     # 2
print(relu(0))     # 0
print(relu(-5))    # 0
```

24.6 `mse(predicted, actual)`

Mean squared error: the average of the squared differences between two same-length lists. It measures how far a model's predictions are from the true values — lower is better.

```
print(mse([1, 2, 3], [1.1, 2.2, 3.1]))
```

```
0.0333333
```

24.7 Putting It Together

Three lines of JTS GO can already describe a tiny prediction pipeline:

```
m1 = matrix([[1, 2], [3, 4]])
m2 = matrix([[5, 6], [7, 8]])
print(matmul(m1, m2))   # [[19, 22], [43, 50]]
print(sigmoid(0))       # 0.5
print(relu(-5))         # 0
print(mse([1, 2, 3], [1.1, 2.2, 3.1]))   # 0.0333333
```

25. The Scroll System

25.1 What Is a Scroll?

A *scroll* is a JTS GO source file that acts as a library module. Scrolls are plain `.jts` files — which means the entire standard library is readable, editable, and copyable JTS GO source code. The name evokes an ancient library: each scroll holds knowledge, and `bring`ing it makes that knowledge available to your program.

25.2 Bringing a Scroll: `bring`

To use a standard-library scroll, write `bring name` near the top of your program. The scroll's functions become available under a namespace with the same name:

```
bring math
bring strings

print(math.factorial(5))            # 120
print(strings.is_palindrome("racecar"))   # true
```

Scroll globals also leak into the program's flat namespace, so `math.factorial(5)` and `factorial(5)` are both valid after `bring math`:

```
bring math
print(math.pi)        # 3.14159   (namespaced)
print(pi)             # 3.14159   (flat)
```

25.3 Sub-Scrolls and Nested Namespaces

Scrolls can be organised into nested namespaces:

```
bring testmath.stats

print(testmath.stats.mean([1, 2, 3, 4, 5]))    # 3
```

25.4 How Scrolls Resolve

When the interpreter encounters `bring name`, it looks for a file named `name.jts` (and sub-scroll paths such as `name/stats.jts`) in this order:

1. The directory containing your program.
2. The installed SDK directory.

3. The current working directory, looking inside a `scrolls/` subfolder.

If the scrolls folder is somewhere unusual, set the `JTS_SCROLLS` environment variable to its location.

> **Tip:** this resolution order means you can override any standard scroll by placing your own file of the same name next to your program — a simple but powerful way to customize the library.

25.5 Bringing Is Idempotent

Bringing the same scroll twice does not double-execute it:

```
bring math
bring math          # safe — no-op the second time
print("OK")
```

25.6 Writing Your Own Scroll

A scroll is just a file of definitions. Create `geometry.jts`:

```
# geometry.jts — a custom scroll
pi = 3.141592653589793

func circle_area(r)
    return pi * r * r
end

func rectangle_area(w, h)
    return w * h
end
```

Now use it from any program:

```
bring geometry

print(geometry.circle_area(2))        # 12.5664
print(rectangle_area(3, 4))           # 12     (flat access)
```

25.7 Importing a File: `import`

The `import` keyword loads and executes another JTS GO file, making its global definitions available to the current program. Unlike `bring`, which resolves bare names from the scroll search path, `import` takes a quoted path and is equivalent to calling the built-in `import_file(path)` function (Chapter 20.3):

```
import "helpers.jts"       # runs helpers.jts — its globals are now visible
print(double(5))           # 10   (defined inside helpers.jts)
```

Use `import` when you want to load a project-local file by path. Use `bring` when you want to load a standard-library scroll by name.

25.8 The Standard Scrolls

Version 2.1 ships fourteen standard scrolls. The remainder of Part IV documents each one with examples and verified output.

Scroll	Focus	Chapter
`text`	Text utilities: slugs, case conversions, ciphers, word tools	26
`strings`	Core string helpers: reverse, palindrome, repetition	27
`stats`	Statistics: mean, median, variance, correlation	28
`seq`	Sequence operations: head, tail, zip, windows, frequencies	29
`lists`	List helpers: flatten, unique, chunk, rotate	30
`numbers`	Number utilities: digit tools, bases, ordinals, formatting	31
`math`	Constants and mathematical helpers	32
`fs`	File-system helpers built on the file built-ins	33
`dates`	Date and time helpers	34
`sets`	Set helpers wrapping the built-in set operations	35
`json`	JSON wrappers around the JSON built-ins	36
`os`	Operating-system helpers	37
`random`	Random helpers: choice, sample, coin flips	38
`time`	Time helpers: stamps, epochs, today's date	39

Note: because scrolls are ordinary JTS GO source, everything in the following chapters can be verified by opening the scroll file itself and reading the implementation. Understanding a scroll's source is an excellent way to improve your own JTS GO.

26. The `text` Scroll

The `text` scroll provides utilities for working with text: converting between naming conventions, building slugs, masking secrets, applying ciphers, and counting words and letters.

```
bring text
```

26.1 Naming-Conversion Helpers

`text.slugify(s)`

Converts text to a URL-friendly slug: lowercase, spaces replaced with hyphens, non-alphanumeric characters removed.

```
print(text.slugify("Hello World, JTS GO!"))   # hello-world-jts-go
```

`text.snake_case(s)`

Converts to `snake_case` (lowercase words joined by underscores).

```
print(text.snake_case("HelloWorld JTS"))      # hello_world_jts
```

`text.kebab_case(s)`

Converts to `kebab-case` (lowercase words joined by hyphens).

```
print(text.kebab_case("HelloWorld JTS"))      # hello-world-jts
```

`text.camel_case(s)`

Converts to `camelCase` (first word lowercase, subsequent words capitalized).

```
print(text.camel_case("hello world jts"))     # helloWorldJts
```

`text.pascal_case(s)`

Converts to `PascalCase` (every word capitalized).

```
print(text.pascal_case("hello world jts"))    # HelloWorldJts
```

26.2 Masking and Abbreviation

`text.mask(s, keep)`

Masks everything except the last `keep` characters with asterisks. Useful for hiding card numbers and secrets.

```
print(text.mask("1234567890", 4))      # ******7890
print(text.mask("password", 3))        # *****ord
```

`text.abbreviate(s, n)`

Keeps the first `n` words and appends `...`.

```
print(text.abbreviate("the quick brown fox jumps", 3))   # the quick brown...
```

`text.truncate_words(s, n)`

Alias-style truncation by word count.

```
print(text.truncate_words("the quick brown fox jumps", 2))  # the quick...
```

26.3 Pluralization

`text.pluralize(word, count)`

Returns the singular word when `count` is 1, and a plural form otherwise. Handles `s`, `x`, `z`, `ch`, `sh` and consonant-plus-`y` endings.

```
print(text.pluralize("cat", 2))      # cats
print(text.pluralize("box", 2))      # boxes
print(text.pluralize("party", 2))    # parties
print(text.pluralize("box", 1))      # box
```

26.4 Word Tools

`text.reverse_words(s)`

```
print(text.reverse_words("the quick brown fox"))   # fox brown quick the
```

`text.initials(s)`

```
print(text.initials("John Fitzgerald Kennedy"))    # JFK
```

`text.normalize_spaces(s)`

```
print(text.normalize_spaces("  a   b  c  "))       # a b c
```

`text.split_words(s)` and `text.is_blank(s)`

`split_words` splits text into words, understanding capitalization boundaries; `is_blank` tests whether a string is empty or whitespace-only.

```
print(text.is_blank("   "))     # true
print(text.count_substring("banana", "an"))   # 2
```

`text.count_vowels(s)` / `text.count_consonants(s)`

```
print(text.count_vowels("hello world"))       # 3
print(text.count_consonants("hello world"))   # 7
```

`text.extract_digits(s)`

```
print(text.extract_digits("abc123xyz456"))    # 123456
```

`text.contains_any(s, chars)`

```
print(text.contains_any("hello", "xyz"))      # false
print(text.contains_any("hello", "oxz"))      # true
```

26.5 Ciphers

`text.rot13(s)`

Rotates each letter 13 places in the alphabet. Applying it twice restores the original text.

```
print(text.rot13("Hello World"))       # Uryyb Jbeyq
print(text.rot13(text.rot13("Hello World")))   # Hello World
```

`text.caesar(s, shift)`

Shifts letters by `shift` places. Negative shifts work.

```
print(text.caesar("Attack at dawn", 3))   # Dwwdfn dw gdzq
```

26.6 Anagram and Character Tools

`text.is_anagram(a, b)`

```
print(text.is_anagram("listen", "silent"))   # true
print(text.is_anagram("hello", "world"))     # false
```

`text.is_isogram(s)`

True when no letter is repeated.

```
print(text.is_isogram("background"))   # true
print(text.is_isogram("letter"))       # false
```

`text.sort_chars(s)`

```
print(text.sort_chars("cba"))          # abc
```

`text.shuffle_chars(s)`

Returns a random reordering of the characters (random output).

```
print(text.shuffle_chars("JTS GO"))    # e.g. J STGO
```

26.7 Formatting

`text.indent(s, spaces)`

```
print(text.indent("line1\nline2", 2))
```

```
  line1
  line2
```

27. The `strings` Scroll

The `strings` scroll provides the core string helpers: reversal, palindrome testing, word counting, repetition, and truncation.

```
bring strings
```

27.1 `strings.reverse(s)`

```
print(strings.reverse("hello"))    # olleh
```

27.2 `strings.is_palindrome(s)`

```
print(strings.is_palindrome("racecar"))   # true
print(strings.is_palindrome("hello"))     # false
```

27.3 `strings.count_words(s)`

```
print(strings.count_words("the quick brown fox"))   # 4
```

27.4 `strings.repeat(s, n)`

```
print(strings.repeat("ab", 3))     # ababab
```

27.5 `strings.truncate(s, n)`

```
print(strings.truncate("a very long string", 8))    # a ver...
```

27.6 `strings.words(s)`

```
print(strings.words("a b c"))     # [a, b, c]
```

27.7 `strings.has_letter(s, ch)`

```
print(strings.has_letter("hello", "e"))   # true
```

27.8 `strings.strip_vowels(s)`

```
print(strings.strip_vowels("hello"))     # hll
```

27.9 `strings.capitalize_words(s)` / `titleize(s)`

```
print(strings.capitalize_words("hello world jts"))   # Hell Worl Jt
print(strings.titleize("hello world"))                # Hell Worl
```

Known limitation: `capitalize_words` / `titleize` currently drop the final character of each word (an off-by-one in the underlying `substring` call). Use `text.pascal_case` (Chapter 26) for correctly capitalized output, or copy the function into your own scroll and fix the `substring(1, len(w) - 1)` call to `substring(1, len(w))`. This is the benefit of a readable standard library: the bug is one line, and you are free to patch it.

27.10 `strings.remove_spaces(s)`

```
print(strings.remove_spaces("a b c"))    # abc
```

28. The `stats` Scroll

The `stats` scroll implements the standard tools of descriptive and inferential statistics: measures of central tendency, measures of spread, percentiles, normalization, and correlation. Every function takes a list of numbers and returns a number or a list.

```
bring stats
```

28.1 Measures of Central Tendency

`stats.mean(data)` — the arithmetic average

```
print(stats.mean([1, 2, 3, 4, 5]))     # 3
```

`stats.median(data)` — the middle value

```
print(stats.median([5, 1, 4, 2, 3]))   # 3
print(stats.median([1, 2, 3, 4]))      # 2.5   (average of the two middle values)
```

`stats.mode(data)` — the most frequent value

```
print(stats.mode([1, 2, 2, 3, 2]))    # 2
```

Weighted, geometric, and harmonic means

```
print(stats.weighted_mean([3, 4, 5], [1, 2, 1]))   # 4
print(stats.geometric_mean([1, 2, 4, 8]))          # 2.82843
print(stats.harmonic_mean([1, 2, 4]))              # 1.71429
```

`stats.trim_mean(data, fraction)`

Removes the given fraction of the smallest and largest values, then averages the rest. It reduces the influence of outliers.

```
print(stats.trim_mean([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 0.1))   # 5.5
```

28.2 Measures of Spread

Variance

Population variance divides by `n`; sample variance divides by `n - 1` (Bessel's correction) and is used when data is a sample of a larger population.

```
data = [2, 4, 4, 4, 5, 5, 7, 9]
print(stats.population_variance(data))   # 4
print(stats.sample_variance(data))       # 4.57143
print(stats.variance(data, true))        # 4.57143   (sample)
print(stats.variance(data, false))       # 4          (population)
```

Standard deviation

```
print(stats.population_stdev(data))   # 2
print(stats.sample_stdev(data))       # 2.13809
print(stats.stdev(data))              # 2.13809   (sample by default)
```

Range and midrange

```
print(stats.range_stat([3, 1, 9, 4]))   # 8    (max - min)
print(stats.midrange([3, 1, 9, 4]))     # 5    ((max + min) / 2)
```

Mean absolute deviation

```
print(stats.mean_absolute_deviation([1, 2, 3, 4]))   # 1
```

28.3 Percentiles

```
print(stats.percentile([1, 2, 3, 4, 5], 50))   # 3
print(stats.quartile1([1, 2, 3, 4, 5, 6, 7, 8]))   # 2.75
print(stats.quartile3([1, 2, 3, 4, 5, 6, 7, 8]))   # 6.25
print(stats.interquartile_range([1, 2, 3, 4, 5, 6, 7, 8]))   # 3.5
```

28.4 Standardization and Normalization

`stats.zscore(x, data)`

How many population standard deviations `x` lies from the mean.

```
print(stats.zscore(12, [10, 12, 14]))   # 0
print(stats.zscore(14, [10, 12, 14]))   # 1.22474
```

`stats.standardize(data)`

Transforms each value to its z-score, giving a list with mean 0 and standard deviation 1.

```
print(stats.standardize([1, 2, 3]))   # [-1.22474, 0, 1.22474]
```

`stats.normalize(data)`

Scales values into the range `[0, 1]`.

```
print(stats.normalize([2, 4, 6, 8]))   # [0, 0.333333, 0.666667, 1]
```

28.5 Correlation

`stats.covariance(xs, ys)`

```
print(stats.covariance([1, 2, 3, 4], [2, 4, 6, 8]))   # 2.5
```

`stats.correlation(xs, ys)`

The Pearson correlation coefficient, always between -1 and 1. Perfect linear relationships give +1 (positive slope) or -1 (negative slope).

```
print(stats.correlation([1, 2, 3, 4], [2, 4, 6, 8]))   # 1
```

28.6 A Complete Analysis

```
bring stats

scores = [85, 92, 78, 91, 84, 90, 76, 88, 95, 81]

print("mean:     " + stats.mean(scores))
print("median:   " + stats.median(scores))
print("stdev:    " + stats.sample_stdev(scores))
print("q1:       " + stats.quartile1(scores))
print("q3:       " + stats.quartile3(scores))
```

```
mean:     86
median:   86.5
stdev:    6.28932
q1:       81.75
q3:       90.75
```

29. The `seq` Scroll

The `seq` scroll provides operations on sequences (lists): functional-style slicing, pairing, windowing, counting, and reshaping. These helpers compose beautifully with the list built-ins from Chapter 17.

```
bring seq
```

29.1 Selecting Parts of a List

```
lst = [1, 2, 3]

print(seq.head(lst))          # 1
print(seq.tail(lst))          # [2, 3]
print(seq.init(lst))          # [1, 2]
print(seq.last_item(lst))     # 3
print(seq.head([]))           # nil   (safe on empty lists)
```

29.2 Slicing: `take`, `drop`, `nth`

```
lst = [1, 2, 3, 4, 5]
print(seq.take(lst, 2))       # [1, 2]
print(seq.drop(lst, 2))       # [3, 4, 5]
print(seq.nth(lst, 1))        # 2
print(seq.nth(lst, -1))       # 5      (negative indexes from the end)
print(seq.nth(lst, 99))       # nil    (out of range is safe)
```

29.3 Pairing: `zip`, `enumerate`, `interleave`

```
print(seq.zip(["a", "b"], [1, 2, 3]))        # [[a, 1], [b, 2]]
print(seq.enumerate(["x", "y", "z"]))         # [[0, x], [1, y], [2, z]]
print(seq.interleave([1, 3], [2, 4, 5]))      # [1, 2, 3, 4, 5]
```

`zip` stops at the shorter list; `interleave` alternates elements and includes the remainder of the longer list.

29.4 Windows and Pairs

```
print(seq.sliding_window([1, 2, 3, 4, 5], 3))
# [[1, 2, 3], [2, 3, 4], [3, 4, 5]]

print(seq.consecutive_pairs([1, 2, 3, 4]))
# [[1, 2], [2, 3], [3, 4]]
```

29.5 Combinations: `product`

```
print(seq.product([1, 2], ["a", "b"]))
# [[1, a], [1, b], [2, a], [2, b]]
```

29.6 Frequencies and Repetition

```
print(seq.frequency(["a", "b", "a", "c", "a", "b"]))
# [[a, 3], [b, 2], [c, 1]]

print(seq.repeat_times([1, 2], 3))     # [1, 2, 1, 2, 1, 2]
print(seq.cycle(["a", "b", "c"], 5))   # [a, b, c, a, b]
```

29.7 Running Calculations

```
print(seq.accumulate([1, 2, 3, 4]))    # [1, 3, 6, 10]   (running totals)
print(seq.differences([1, 3, 6, 10]))  # [2, 3, 4]       (successive gaps)
```

29.8 Inserting Separators

```
print(seq.intersperse(["a", "b", "c"], "-"))   # [a, -, b, -, c]
```

29.9 Shuffling a Copy

```
print(seq.shuffle_copy([1, 2, 3, 4, 5]))   # a random ordering (original untouched)
```

29.10 Building a Histogram of Letter Frequencies

```
bring seq

func letter_freq(text)
    return seq.frequency(text.split(""))
end

print(letter_freq("hello"))
# [[h, 1], [e, 1], [l, 2], [o, 1]]
```

30. The `lists` Scroll

The `lists` scroll adds higher-level list helpers on top of the built-in list operations from Chapter 17: flattening, de-duplication, reversing, chunking, rotation, and arithmetic over lists.

```
bring lists
```

30.1 `lists.flatten(lst)`

Flattens nested lists recursively, producing a single flat list.

```
print(lists.flatten([[1, 2], [3, [4, 5]]]))
# [1, 2, 3, 4, 5]
```

30.2 `lists.unique(lst)`

Returns a copy with duplicate values removed, preserving first-seen order.

```
print(lists.unique([1, 2, 2, 3, 1, 4, 2]))   # [1, 2, 3, 4]
```

30.3 `lists.reversed(lst)`

```
print(lists.reversed([1, 2, 3]))   # [3, 2, 1]
```

30.4 `lists.chunk(lst, size)`

Divides a list into consecutive sub-lists of at most `size` elements.

```
print(lists.chunk([1, 2, 3, 4, 5], 2))       # [[1, 2], [3, 4], [5]]
```

30.5 `lists.rotate(lst, k)`

Rotates elements *right* by `k` positions. Negative values rotate left; `k` is reduced modulo the list length.

```
print(lists.rotate([1, 2, 3, 4, 5], 1))      # [5, 1, 2, 3, 4]
print(lists.rotate([1, 2, 3, 4, 5], -1))     # [2, 3, 4, 5, 1]
print(lists.rotate([1, 2, 3, 4, 5], 6))      # [5, 1, 2, 3, 4]
```

30.6 `lists.median(sorted_list)`

Computes the median of an already-sorted list. Note the contract: the caller is responsible for sorting.

```
print(lists.median([1, 2, 3, 4, 5]))       # 3
print(lists.median([1, 2, 3, 4]))          # 2.5
print(lists.median([]))                    # 0
```

30.7 `lists.first`, `lists.last`

```
print(lists.first([7, 8, 9]))   # 7
print(lists.last([7, 8, 9]))    # 9
```

30.8 Arithmetic Helpers

```
print(lists.sum_squares([1, 2, 3]))              # 14
print(lists.count_occurrences([1, 2, 2, 3], 2))  # 2
```

31. The `numbers` Scroll

The `numbers` scroll provides number utilities: numeric validation, digit extraction, base conversion, percentages, formatting, ordinals, and bit analysis.

```
bring numbers
```

31.1 Testing and Coercion

```
print(numbers.is_numeric("12345"))   # true
print(numbers.is_numeric("-42"))     # true
print(numbers.is_numeric("12a4"))    # false
print(numbers.is_numeric(""))        # false
print(numbers.is_integer(7))         # true
print(numbers.is_integer(7.5))       # false
print(numbers.is_whole(3.0))         # true
```

31.2 Digit Tools

```
print(numbers.digit_count(12345))    # 5
print(numbers.digit_count(0))        # 1
print(numbers.digits(1234))          # [1, 2, 3, 4]
print(numbers.from_digits([1, 2, 3]))  # 123
print(numbers.reverse_digits(1234))  # 4321
```

31.3 Base Conversion

```
print(numbers.to_binary(42))     # 101010
print(numbers.to_octal(42))      # 52
print(numbers.to_hex(255))       # ff
print(numbers.to_hex(-255))      # -ff

print(numbers.from_binary("101010"))   # 42
print(numbers.from_octal("52"))        # 42
print(numbers.from_hex("ff"))          # 255
print(numbers.from_base("ff", 16))     # 255
print(numbers.from_base("2", 2))       # nil   (2 is not a valid binary digit)
```

31.4 Percentages

```
print(numbers.percent(30, 200))        # 15
print(numbers.percent_change(50, 60))  # 20
```

31.5 Formatting

```
print(numbers.round_to(3.14159, 2))   # 3.14
print(numbers.with_commas(1234567))   # 1,234,567
print(numbers.with_commas(-9876543))  # -9,876,543
```

31.6 Ordinals

```
print(numbers.ordinal(1))    # 1st
print(numbers.ordinal(2))    # 2nd
print(numbers.ordinal(3))    # 3rd
print(numbers.ordinal(11))   # 11th   (the special 11/12/13 case)
print(numbers.ordinal(21))   # 21st
```

31.7 Bits

```
print(numbers.is_power_of_two(16))   # true
print(numbers.is_power_of_two(18))   # false
print(numbers.count_bits(13))        # 3   (13 = 1101 in binary)
```

31.8 A Guessing-Game Check

```
bring numbers

secret = 42
guess = 42
if guess == secret
    print("You guessed " + numbers.ordinal(guess) + " correct!")
end
```

```
You guessed 42nd correct!
```

32. The `math` Scroll

The `math` scroll provides constants and mathematical helpers built on the `math()` built-in from Chapter 15 and the core numeric built-ins.

```
bring math
```

32.1 Constants

```
print(math.pi)       # 3.14159
print(math.e)        # 2.71828
print(math.tau)      # 6.28319
print(math.phi)      # 1.61803
print(math.sqrt2)    # 1.41421
```

32.2 Logarithms (any base)

```
print(math.log2(8))    # 3
print(math.log10(100)) # 2
```

32.3 Geometry

```
print(math.hypot(3, 4))       # 5
print(math.degrees(pi))       # 180
print(math.radians(180))      # 3.14159
```

32.4 Sign, Clamping, and Interpolation

```
print(math.sign(-5))   # -1
print(math.sign(0))    # 0
print(math.sign(7))    # 1
print(math.clamp(150, 0, 100))   # 100
print(math.lerp(0, 10, 0.5))     # 5
```

32.5 Parity and Number Theory

```
print(math.is_even(4))    # true
print(math.is_odd(5))     # true
print(math.factorial(5))  # 120
print(math.gcd(12, 18))   # 6
print(math.lcm(4, 6))     # 12
print(math.is_prime(17))  # true
print(math.is_prime(18))  # false
```

32.6 Averages and Digits

```
print(math.avg([10, 20, 30]))   # 20
print(math.midpoint(2, 6))      # 4
print(math.digit_sum(12345))    # 15
```

33. The `fs` Scroll

The `fs` scroll wraps the file built-ins (Chapter 20) with friendly helpers for paths, line reading, and file inspection.

```
bring fs
```

33.1 Reading and Writing

```
print(fs.write("demo.txt", "hello\nworld\n"))   # true
print(fs.read("demo.txt"))                      # hello\nworld\n
print(fs.read_lines("demo.txt"))                # [hello, world]
print(fs.count_lines("demo.txt"))               # 2
print(fs.first_line("demo.txt"))                # hello
print(fs.last_line("demo.txt"))                 # world
print(fs.append_text("demo.txt", "again\n"))    # true
print(fs.read("demo.txt"))                      # hello\nworld\nagain\n
```

Tip: `read_lines` also drops the trailing empty line created by a final newline, which makes line counting exact.

33.2 File Facts

```
print(fs.exists("demo.txt"))          # true
print(fs.file_size("demo.txt"))       # number of bytes
print(fs.is_empty_file("demo.txt"))   # false
```

33.3 Path Tools

```
print(fs.basename("C:/docs/report.txt"))    # report.txt
print(fs.dirname("C:/docs/report.txt"))     # C:/docs
print(fs.ext("report.txt"))                 # .txt   (includes the dot)
print(fs.with_extension("report.txt", ".csv"))  # report.csv
print(fs.join_path("a", "b"))               # a/b
print(fs.join_path("a/", "b"))              # a/b
```

Note: `fs.ext` returns the extension *with* its leading dot, so `"report.txt"` gives `".txt"`. Compare with the language-agnostic convention of other languages, which usually omit the dot.

33.4 Copying

```
print(fs.copy_text("demo.txt", "copy.txt"))   # true
print(fs.read("copy.txt"))                    # hello\nworld\nagain\n
```

33.5 Word Count

```
print(fs.word_count("scrolls/text.jts"))   # number of words in the file
```

> **Note:** the exact word count depends on the file contents on your machine. The functions are deterministic; only the underlying data varies.

34. The `dates` Scroll

The `dates` scroll provides date helpers built on the time built-ins (Chapter 22). Timestamps are epoch seconds (as returned by `now()`).

```
bring dates
```

34.1 Readable Timestamps

```
print(dates.unix())          # current epoch seconds
print(dates.today())         # YYYY-MM-DD
print(dates.time_str())      # HH:MM:SS
print(dates.stamp())         # YYYY-MM-DD HH:MM:SS
print(dates.fmt(now(), "%A, %B %d"))   # e.g. Tuesday, August 04
```

34.2 Date Arithmetic

```
t = now()
print(dates.add_seconds(t, 30))    # t + 30
print(dates.add_minutes(t, 5))     # t + 300
print(dates.add_hours(t, 1))       # t + 3600
print(dates.add_days(t, 1))        # t + 86400
print(dates.add_weeks(t, 1))       # t + 604800
print(dates.add_months(t, 2))      # t + 5184000
print(dates.add_years(t, 1))       # t + 31557600
```

> **Note:** months and years are approximate (30 days and 365.25 days). For calendar-accurate month arithmetic, work with `strftime` components as shown in section 34.5.

34.3 Differences Between Two Stamps

```
a = now()
b = dates.add_hours(a, 2)
print(dates.seconds_between(a, b))   # 7200
print(dates.minutes_between(a, b))   # 120
print(dates.hours_between(a, b))     # 2
print(dates.days_between(a, b))      # 0.0833333
```

34.4 Weekdays

```
print(dates.day_of_week(now()))     # 1..7  (1 = Monday ... 7 = Sunday)
print(dates.day_name(now()))        # e.g. Tue
print(dates.day_name_full(now()))   # e.g. Tuesday
print(dates.is_weekend(now()))      # true or false
print(dates.is_weekday(now()))      # the opposite
```

34.5 Decomposing and Rebuilding

```
d = dates.date_parts(now())
print(d["year"])     # e.g. 2026
print(d["month"])    # e.g. 8
print(d["day"])      # e.g. 4
print(d["hour"])     # current hour
print(d["weekday"])  # 1..7

print(dates.start_of_day(now()))    # midnight today (epoch)
print(dates.end_of_day(now()))      # 23:59:59 today (epoch)
print(dates.start_of_hour(now()))   # top of the current hour
print(dates.start_of_minute(now())) # top of the current minute
```

34.6 Calendar Facts

```
print(dates.is_leap_year(2024))   # true
print(dates.is_leap_year(2025))   # false
print(dates.month_days(2026, 2))  # 28
print(dates.month_days(2024, 2))  # 29
print(dates.day_of_year(now()))   # e.g. 216
print(dates.iso_week(now()))      # e.g. 32
```

34.7 Human-Readable Durations

```
print(dates.human_duration(45))       # 45s
print(dates.human_duration(90))       # 1m 30s
print(dates.human_duration(3600))     # 1h
print(dates.human_duration(90061))    # 1d 1h 1m 1s
```

34.8 A Countdown Program

```
bring dates

launch = dates.add_days(now(), 14)
print("T-minus " + dates.human_duration(launch - now()) + " to launch")
```

```
T-minus 14d to launch
```

35. The `sets` Scroll

The `sets` scroll provides friendly helpers around the built-in set operations from Chapter 18: construction, algebra, membership tests, and batch mutation.

```
bring sets
```

35.1 Construction

```
print(sets.from_list([1, 2, 2, 3]))       # {1, 2, 3}
print(sets.from_string("hello"))          # {e, h, l, o}
print(sets.singleton(7))                  # {7}   (works where {7} alone fails — see 4.x)
```

35.2 Set Algebra

```
a = sets.from_list([1, 2, 3])
b = sets.from_list([3, 4, 5])

print(sets.union(a, b))              # {1, 2, 3, 4, 5}
print(sets.intersection(a, b))       # {3}
print(sets.difference(a, b))         # {1, 2}
print(sets.symmetric_difference(a, b))   # {1, 2, 4, 5}
```

35.3 Membership and Size

```
s = sets.from_list([1, 2, 3])
print(sets.has_item(s, 2))    # true
print(sets.size(s))           # 3
print(sets.is_empty(sets.from_list([])))   # true
```

35.4 Relation Tests

```
a = sets.from_list([1, 2])
b = sets.from_list([1, 2, 3])
c = sets.from_list([4, 5])

print(sets.is_subset(a, b))    # true
print(sets.is_superset(b, a))  # true
print(sets.is_disjoint(a, c))  # true
print(sets.equals(a, sets.from_list([2, 1])))   # true
```

35.5 Batch Testing and Mutation

```
s = sets.from_list([1, 2, 3])
print(sets.has_any(s, [9, 2]))     # true
print(sets.has_all(s, [1, 3]))     # true
print(sets.add_all(s, [4, 5]))     # {1, 2, 3, 4, 5}
print(sets.remove_all(s, [2, 3]))  # {1, 4, 5}
```

35.6 `sets.add_item` and `sets.remove_item`

These mutate the set in place and return it, so they can be chained:

```
s = sets.from_list([1])
sets.add_item(s, 2)
sets.remove_item(s, 1)
print(s)     # {2}
```

36. The `json` Scroll

The `json` scroll wraps the JSON built-ins (Chapter 21) with short names and file helpers.

```
bring json
```

36.1 `json.parse` and `json.stringify`

```
data = json.parse("{\"name\": \"Ada\", \"scores\": [9, 8, 10]}")
print(data["name"])                # Ada
print(data["scores"][0])           # 9

print(json.stringify({"ok": true, "n": 42}))    # {"ok":true,"n":42}
```

36.2 `json.pretty`

Alias of `stringify`, available for readability when you want to signal intent.

```
print(json.pretty([1, "two", false, nil]))    # [1,"two",false,null]
```

36.3 `json.load` and `json.save` — persistence

The pair makes saving and restoring program state a two-line task:

```
bring json

config = { "theme": "dark", "volume": 0.7 }
json.save("config.json", config)

loaded = json.load("config.json")
print(loaded["theme"])     # dark
print(loaded["volume"])    # 0.7
```

> **Tip:** `json.save` is the recommended way to persist application data. Combined with the `fs` scroll's helpers, you have a complete file-storage stack.

37. The `os` Scroll

The `os` scroll gives environment, directory, argument, and file helpers under one namespace.

```
bring os
```

37.1 Environment

```
print(os.get_env("PATH"))      # the PATH value, or nil
print(os.current_dir())        # the working directory
print(os.program_args())       # the command-line arguments
```

37.2 Files

```
print(os.file_write("note.txt", "hello\n"))    # true
print(os.file_read("note.txt"))                # hello\n
```

38. The `random` Scroll

The `random` scroll wraps the random built-ins (Chapter 19) with higher-level sampling helpers.

```
bring random
```

38.1 Basics

```
print(random.random())          # a float in [0, 1)
print(random.int_between(1, 6))    # an integer in [1, 6]
print(random.coin_flip())       # true or false
```

38.2 Picking From a List

```
deck = ["king", "queen", "jack", "ace"]
print(random.choice(deck))           # one random element
print(random.choice([]))             # nil (empty list is safe)
print(random.choices(deck, 2))       # two elements, replacement allowed
print(random.sample(deck, 2))        # two elements, no duplicates
```

> **Note:** `choices` may return repeats; `sample` shuffles a copy and takes distinct elements.

38.3 A Dice Roller

```
bring random

func roll_dice(n)
    results = []
    i = 0
    while i < n
        results.append(random.int_between(1, 6))
        i = i + 1
    end
    return results
end

print(roll_dice(3))
```

```
[5, 2, 6]       (a typical run; your values will differ)
```

38.4 Reproducible Experiments

Like the built-in `rand`, the scroll helpers respect `seed`. Setting the same seed produces the same sequence, which is essential when testing programs that use randomness:

```
bring random
seed(7)
print(random.choice(["a", "b", "c"]))   # e.g. c
seed(7)
print(random.choice(["a", "b", "c"]))   # c again — identical
```

39. The `time` Scroll

The `time` scroll provides simple, named wrappers for the time built-ins (Chapter 22): stamps, epochs, and sleeping.

```
bring time
```

39.1 Timestamps

```
print(time.today())        # YYYY-MM-DD
print(time.clock_time())   # HH:MM:SS
print(time.stamp())        # YYYY-MM-DD HH:MM:SS
print(time.stamp_utc())    # YYYY-MM-DDTHH:MM:SSZ (UTC)
print(time.epoch())        # epoch seconds
```

```
2026-08-04
14:32:07
2026-08-04 14:32:07
2026-08-04T14:32:07Z
1788510727
```

(Values shown are examples; the actual output depends on the current moment.)

39.2 Sleeping

```
print("waiting...")
time.sleep_ms(250)
print("done")          # printed about a quarter-second later
```

`time.sleep_ms` accepts milliseconds, which is more convenient than the built-in `sleep`'s seconds.

39.3 A Simple Stopwatch

```
bring time

start = now()
count = 0
while count < 100000
    count = count + 1
end
elapsed = now() - start

print("counted in " + time.clock_time())
print("elapsed ms: " + str(elapsed * 1000))
```

```
counted in 14:33:02
elapsed ms: 1.5        (approximate — depends on your machine)
```

40. Machine Learning with JTS GO

JTS GO ships machine-learning primitives as built-ins: `tensor`, `matrix`, `matmul`, `sigmoid`, `relu`, and `mse` (Chapters 15 and 24). This chapter puts them together to train a real neural network: a 2-3-1 multi-layer perceptron that learns the XOR function, the classic "hello world" of machine learning.

40.1 The XOR Problem

XOR is not linearly separable: you cannot draw a single straight line that separates true from false outputs. A single-layer perceptron cannot learn it; a network with one hidden layer can. The dataset is tiny:

Input A	Input B	XOR output
0	0	0
0	1	1
1	0	1
1	1	0

40.2 Network Architecture

- **Input layer:** 2 neurons (the two bits).
- **Hidden layer:** 3 neurons, each with a sigmoid activation.
- **Output layer:** 1 neuron with a sigmoid activation, producing a value in `[0, 1]`.

Each layer has an extra row of weights for the *bias* — an input that is always 1, giving each neuron a constant offset it can learn. The weight matrices are `w1` (3×3) and `w2` (4×1).

40.3 Random Initialization

Weights start as small random values near zero. The `for r in 0 to rows` loop form was introduced in Chapter 8:

```
func init_layer(rows, cols)
    layer = []
    for r in 0 to rows
        row = []
        for c in 0 to cols
            append(row, rand() - 0.5)
        end
        append(layer, row)
    end
    return layer
end

w1 = init_layer(3, 3)   # inputs+1 rows, 3 hidden neurons
w2 = init_layer(4, 1)   # hidden+1 rows, 1 output neuron
```

40.4 The Forward Pass

Each hidden neuron computes a weighted sum of the inputs plus its bias, then applies `sigmoid`. The output neuron does the same using the hidden activations:

```
func forward(x)
    a1 = []
    for j in 0 to n_hidden
        z = w1[n_in][j]
        for i in 0 to n_in
            z = z + x[i] * w1[i][j]
        end
        append(a1, sigmoid(z))
    end

    z = w2[n_hidden][0]
    for j in 0 to n_hidden
        z = z + a1[j] * w2[j][0]
    end
    out = sigmoid(z)
    return [out, a1]
end
```

40.5 The Backward Pass (Gradient Descent)

For each training sample, the network computes its output, measures the error, and nudges every weight against the gradient of that error. The derivative of the sigmoid `s` is `s(1-s)`, which is why `d_out = diff * out * (1 - out)` appears in the code:

```
diff = out - t
d_out = diff * out * (1 - out)

w2[n_hidden][0] = w2[n_hidden][0] - lr * d_out
for j in 0 to n_hidden
    w2[j][0] = w2[j][0] - lr * d_out * a1[j]
end
```

The hidden layer's weight updates chain the gradient backward through the output layer:

```
for j in 0 to n_hidden
    dz1 = d_a1[j] * a1[j] * (1 - a1[j])
    w1[n_in][j] = w1[n_in][j] - lr * dz1
    for i in 0 to n_in
        w1[i][j] = w1[i][j] - lr * dz1 * x[i]
    end
end
```

The learning rate `lr = 1.0` controls how large each step is. Too large, and training diverges; too small, and it crawls. A common debugging technique is printing the loss every few hundred epochs:

```
if e % 500 == 0
    print("epoch " + e + "   loss " + round(total_loss / 4 * 1000) / 1000)
end
```

40.6 Complete Program and Verified Output

```
# A 2-3-1 multi-layer perceptron trained on XOR with gradient descent.
inputs  = [[0, 0], [0, 1], [1, 0], [1, 1]]
targets = [[0], [1], [1], [0]]

n_in = 2
n_hidden = 3
n_out = 1
lr = 1.0
n_samples = len(inputs)

func init_layer(rows, cols)
    layer = []
    for r in 0 to rows
        row = []
        for c in 0 to cols
            append(row, rand() - 0.5)
        end
        append(layer, row)
    end
    return layer
end

w1 = init_layer(n_in + 1, n_hidden)
w2 = init_layer(n_hidden + 1, n_out)

func forward(x)
    a1 = []
    for j in 0 to n_hidden
        z = w1[n_in][j]
        for i in 0 to n_in
            z = z + x[i] * w1[i][j]
        end
        append(a1, sigmoid(z))
    end
    z = w2[n_hidden][0]
    for j in 0 to n_hidden
        z = z + a1[j] * w2[j][0]
    end
    out = sigmoid(z)
    return [out, a1]
end

func train(epochs)
    for e in 0 to epochs
        total_loss = 0
        for s in 0 to n_samples
            x = inputs[s]
            t = targets[s][0]
            f = forward(x)
            out = f[0]
            a1 = f[1]
            diff = out - t
            total_loss = total_loss + diff * diff
            d_out = diff * out * (1 - out)
```

```
            d_a1 = []
            for j in 0 to n_hidden
                append(d_a1, d_out * w2[j][0])
            end
            w2[n_hidden][0] = w2[n_hidden][0] - lr * d_out
            for j in 0 to n_hidden
                w2[j][0] = w2[j][0] - lr * d_out * a1[j]
            end
            for j in 0 to n_hidden
                dz1 = d_a1[j] * a1[j] * (1 - a1[j])
                w1[n_in][j] = w1[n_in][j] - lr * dz1
                for i in 0 to n_in
                    w1[i][j] = w1[i][j] - lr * dz1 * x[i]
                end
            end
        end
        if e % 500 == 0
            print("epoch " + e + "   loss " + round(total_loss / 4 * 1000) / 1000)
        end
    end
end

train(5000)

print("--- Trained network on XOR ---")
preds = []
for s in 0 to n_samples
    out = forward(inputs[s])[0]
    append(preds, out)
    print(str(inputs[s]) + "  ->  " + round(out * 1000) / 1000 + "   (target " + targets[s]
[0] + ")")
end

target_list = [0, 1, 1, 0]
print("final MSE: " + round(mse(preds, target_list) * 1000) / 1000)
```

The output below was captured from a real run of this program. Because the initial weights are random, your loss values will differ slightly, but the trajectory — a loss falling from about 0.28 toward 0 and predictions approaching the XOR targets — will be the same.

```
epoch 0    loss 0.285
epoch 500    loss 0.169
epoch 1000    loss 0.138
epoch 1500    loss 0.135
epoch 2000    loss 0.135
epoch 2500    loss 0.134
epoch 3000    loss 0.134
epoch 3500    loss 0.134
epoch 4000    loss 0.133
epoch 4500    loss 0.007
--- Trained network on XOR ---
[0, 0]  ->  0.034   (target 0)
[0, 1]  ->  0.963   (target 1)
[1, 0]  ->  0.946   (target 1)
[1, 1]  ->  0.061   (target 0)
final MSE: 0.002
```

40.7 Making Training Reproducible

Call `seed(n)` at the top of the program to lock in the random sequence (Chapter 19). With a fixed seed you get identical weights, identical loss curves, and identical predictions every run — invaluable when you compare hyperparameters:

```
seed(1)            # reproducible training run
train(5000)
```

40.8 Interpreting the Results

A network that has learned XOR produces outputs near 0 for `[0,0]` and `[1,1]`, and near 1 for `[0,1]` and `[1,0]`. The final MSE of about 0.002 means the average squared error is small — the network is effectively solving the problem. The same pattern generalizes to larger networks: more hidden neurons, more layers, and richer activation functions all follow the machinery shown here.

41. Web Development with JTS GO

JTS GO includes a built-in HTTP server, so a web page can be served directly from a `.jts` program with no external dependencies. This chapter builds a complete single-page website using the same techniques the official JTS GO site uses.

41.1 The Server Built-ins

Three built-ins cover everything (Chapter 23):

Built-in	Purpose
`http_server(port, html)`	Creates a server object bound to a port, serving the given HTML.
`http_start(srv)`	Starts the server; the program stays alive serving requests.
`http_request(url)`	Makes an outbound HTTP request and returns the response.

41.2 Building a Page From Parts

Real pages mix static markup with values computed at runtime. The cleanest approach is to build the HTML string in pieces and join them — exactly how the official site works:

```
head = "<!DOCTYPE html> <html> <head> <title>JTS GO</title> <style> " +
       "body { font-family: Segoe UI, sans-serif; background: #0a0a1a; color: #e0e0e0; } " +
       ".hero { text-align: center; padding: 80px 20px; } " +
       ".hero h1 { color: #00ff88; font-size: 3em; } " +
       ".card { background: #16213e; border-radius: 12px; padding: 25px; } " +
       "</style> </head> <body> "

hero = "<div class='hero'> <h1>JTS GO</h1> <p>Web apps in your language.</p> </div> "

time_card = "<div class='card'> <h3>Current time</h3> <p>" + time.today() + " " +
time.clock_time() + "</p> </div> "

footer = "<footer> Made with JTS GO </footer> </body> </html>"

html = head + hero + time_card + footer

srv = http_server(8080, html)
print("Serving at http://localhost:8080")
http_start(srv)
```

Notice how the "Current time" card embeds values computed by the `time` scroll (Chapter 39): because the page is assembled by running code, its content can be dynamic.

41.3 The Official Site Pattern

The repository's `tests/website.jts` example assembles a full marketing site from seven string parts (`part1` through `part7`):

```
html = part1 + part2 + part3 + part4 + part5 + part6 + part7
print("Starting JTS GO website on port 8080...")
srv = http_server(8080, html)
http_start(srv)
```

Each part holds one page section: the document head and styles, the hero banner, a features grid, a code example, an ML demo, an install box, and the footer. Concatenating keeps each string short and readable.

41.4 Running the Server

```
jts tests/website.jts
```

```
Starting JTS GO website on port 8080...
Open http://localhost:8080 in your browser
```

Leave the program running and open the URL. The server keeps serving until you stop the program (Ctrl+C).

> **Note:** `http_server` binds the local port and serves the provided HTML string. It is designed for development and demonstration; for production deployments you would normally put a reverse proxy in front.

41.5 Client Requests: `http_request`

In the opposite direction, `http_request(url)` fetches a resource and returns its content:

```
data = http_request("https://example.com/")
print(len(data))       # the number of characters returned
```

This turns JTS GO into a scripting tool for REST APIs, web scraping, and automated health checks. Combine it with `json.parse` (Chapter 21) to consume JSON APIs:

```
bring json
resp = http_request("https://api.example.com/status")
print(json.parse(resp))
```

42. Debugging and Best Practices

This chapter collects the habits and techniques that separate working programs from programs that merely exist. Every topic builds on material from earlier chapters.

42.1 The Debug Loop

Bugs are inevitable; the trick is finding them quickly. JTS GO's error messages name the line and the file:

```
print(1 / 0)
```

```
STDERR: Division by zero.
[line 1] in script
Runtime error.
```

Use the line number to jump straight to the offending statement, then simplify: reproduce the problem in the smallest possible program, print the intermediate values, and fix one thing at a time.

42.2 Reading Stack Traces

When an error occurs inside a function, JTS GO reports the call chain:

```
func inner(x)
    return 10 / x
end
func outer()
    return inner(0)
end
print(outer())
```

```
STDERR: Division by zero.
[line 2] in inner()
[line 5] in outer()
[line 7] in script
Runtime error.
```

Read traces from the top: the deepest frame is where the error happened, and each line below is the caller that invoked it. The bottom line, `in script`, is the top-level program.

42.3 Debugging With `print`

The oldest tool is still the best. Print values at the boundary between suspicious code and the rest of the program:

```
func area(w, h)
    print("area: w=" + w + " h=" + h)
    return w * h
end
```

Once you confirm the inputs, remove the print. If you forget, the `assert` built-in (Chapter 8) can carry the check permanently:

```
func area(w, h)
    assert(w > 0 and h > 0, "dimensions must be positive")
    return w * h
end
```

42.4 Gotchas to Watch For

JTS GO deliberately differs from other languages in a few places. Knowing these will save hours:

Gotcha	Detail	See
Empty lists are truthy	`if []` is true; check `len(lst) == 0` instead	5.x
Dict subscript assignment fails	`d["k"] = v` is a runtime error; use `d.update()` or `update(d, ...)`	11.x
Single-element set literal	`{1}` is a compile error (parsed as a dict key); use `set([1])`	4.x
Block-close keywords	Every `if`/`for`/`while`/`func`/`class` ends with `end`	4.x
Division by zero	Raises an error; guard with a check before dividing	7.x
Integer division floors	`-10 // 3` is `-4`, not `-3`	7.x

42.5 Code Organization

- **Keep functions small.** A function that does one thing is easy to test and reuse.
- **Name things honestly.** `score` beats `s`; `total_price` beats `tp`.
- **Put reusable code in scrolls.** Anything used by more than one program belongs in a `.jts` library file that you `bring` (Chapter 25).
- **Validate input early.** Check `len()` before indexing, test division guards before dividing, and validate JSON before indexing keys.
- **Prefer return values over globals.** Passing data in and out of functions keeps behavior predictable (Chapter 6).

42.6 Testing With `assert`

A tiny assertion harness can catch regressions. Save it as your own scroll and reuse it:

```
# testing.jts — a minimal test harness
func check(desc, got, expected)
    if got == expected
        print("PASS  " + desc)
    else
        print("FAIL  " + desc + "  (got " + str(got) + ", expected " + str(expected) + ")")
    end
end
```

```
bring testing

check("factorial(5)", factorial(5), 120)
check("gcd(12, 18)", gcd(12, 18), 6)
```

```
PASS  factorial(5)
PASS  gcd(12, 18)
```

42.7 Performance Notes

- **Precompute loop bounds.** Store `len(lst)` in a variable when iterating by index; it avoids repeated calls.
- **Reuse strings with `join`.** Building a string with `+` in a hot loop is slower than collecting parts and joining once.
- **Prefer built-ins.** The VM's native functions (`sort`, `append`, `set_union`, and friends) are far faster than hand-written loops.
- **Understand complexity.** A nested loop over a list is $O(n^2)$; the set built-ins give O(1) membership. Choose the data structure that fits (Chapter 11).

42.8 A Checklist Before You Finish

1. Do all blocks have their matching `end`?
2. Are every variable's possible values covered (empty lists, zero, negative numbers, nil)?
3. Does every function return on all paths?
4. Are side effects (file writes, global mutation) intentional?
5. Does the program run cleanly from a fresh directory, with the scrolls it needs available?

42.9 Further Learning

The fastest way to grow is to read the interpreter itself:

- `scrolls/*.jts` — the entire standard library is readable JTS GO source; study how each function is written.
- `tests/*.jts` — the language's own test suite doubles as a catalogue of idiomatic programs.
- `examples/*.jts` — complete programs including the neural network of Chapter 40.

And when you are ready to look inside the machine, the `src/compiler` and `src/vm` directories of the JTS GO source tree hold the scanner, parser, bytecode, and virtual machine that execute every program in this book.

Appendix A. Keywords

JTS GO has a compact set of reserved words. Identifiers cannot use them. This list is the complete set, extracted from the language's scanner.

Keyword	Purpose	Chapter
`and`	Logical conjunction	7
`append`	Add an element to a list	17
`assert`	Runtime check	8
`bool`	Type name (boolean)	5
`bring`	Load a scroll (module)	25
`break`	Exit a loop	8
`catch`	Handle a thrown value	13
`class`	Declare a class	10
`continue`	Skip to the next iteration	8
`del`	Delete a list element	17
`elif`	Else-if branch	8
`else`	Fallback branch	8
`end`	Close any block	4
`extends`	Inherit from a class	10
`false`	Boolean literal	5
`finally`	Always-run block after try	13
`float`	Type name (floating point)	5
`for`	Iterate over a range or list	8
`func`	Declare a function	9
`http`	Web/server namespace	23
`if`	Conditional branch	8
`import`	Import a file	25
`in`	Membership test; loop variable binder	7
`input`	Read a line from the user	14

`int`	Type name; convert to integer	5
`is`	Identity / type test	7
`len`	Length of a value	14
`list`	Type name; convert to list	5
`matrix`	Create a matrix	24
`model`	AI model construct	24
`new`	Instantiate an object	10
`nil`	Null literal	5
`not`	Logical negation	7
`number`	Type name; convert to number	5
`or`	Logical disjunction	7
`predict`	AI prediction construct	24
`print`	Write to the console	14
`request`	HTTP request construct	23
`response`	HTTP response construct	23
`return`	Return from a function	9
`self`	The current object	10
`server`	Web server construct	23
`set`	Type name; convert to set	5
`string`	Type name; convert to string	5
`super`	The parent class	10
`tensor`	Create a tensor	24
`throw`	Raise an error value	13
`to`	Range loop syntax `for x in 0 to n`	8
`true`	Boolean literal	5
`try`	Begin a protected block	13
`type`	Type of a value	14
`var`	Declare a variable	6
`while`	Loop while a condition holds	8

`yield`	Produce a value from a generator	9

> **Note:** many of these keywords are also ordinary built-in function names (for example `print`, `len`, and `input`), so "keyword" and "built-in" overlap in JTS GO. You can always recognize them by their syntax role: `if` introduces a block, `print` is a callable.

Appendix B. Operator Precedence

From lowest to highest precedence. Operators on the same row associate left-to-right. Higher rows bind more tightly.

Precedence	Operators	Associativity
Assignment	`=`	right
Logical or	`or`	left
Logical and	`and`	left
Equality	`== !=`	left
Comparison	`< > <= >=`	left
Bitwise or	`|`	left
Bitwise xor	`^`	left
Bitwise and	`&`	left
Shift	`<< >>`	left
Term	`+ -`	left
Factor	`* / // %`	left
Unary	`-x not x`	right
Call	`f(x) x[i] o.m()`	left
Primary	literals, identifiers, `(expr)`	—

> **Tip:** when in doubt, add parentheses. `(a + b) * c` never surprises the next reader, and the interpreter follows the same rules the table states.

Appendix C. Built-in Quick Reference

A complete index of the core built-ins described in Part III, with their signatures. Chapter references point to the full documentation and examples.

C.1 Console and I/O

Built-in	Description	See
`print(...)`	Print values to the console	14
`input(prompt?)`	Read a line of text	14
`args()`	Command-line arguments	20
`cwd()`	Current working directory	20
`env(name)`	Environment variable value	20
`exit(code?)`	Terminate the program	20

C.2 Values and Types

Built-in	Description	See
`type(x)`	Type name as a string	14
`len(x)` / `length(x)`	Length of a list, string, dict, or set	14
`str(x)` / `string(x)`	Convert to a string	14
`number(x)` / `float(x)`	Convert to a number	14
`int(x)`	Convert to an integer (truncating)	14
`bool(x)`	Convert to a boolean	14

C.3 Math

Built-in	Description	See
`math(op, x)`	sin, cos, tan, sqrt, abs, log, exp, pow, floor, ceil, round	15
`min(a, b)` / `min(...)`	Smallest value	15
`max(a, b)` / `max(...)`	Largest value	15

`sum(list)`	Total of a list	15
`range(start?, end?)`	Build a list of integers	15

C.4 Strings

Built-in	Description	See
`upper` / `lower`	Change case	16
`trim`, `ltrim`, `rtrim`	Remove surrounding whitespace	16
`pad_left` / `pad_right`	Pad to a width	16
`contains`, `starts_with`, `ends_with`	Search	16
`find`, `index`	Position of a sub-string	16
`is_digit`, `is_alpha`, `is_alnum`, `is_upper`, `is_lower`	Character tests	16
`split`, `splitlines`	Break into pieces	16
`join(sep, list)`	Combine pieces	16
`substring(start, end?)`	Slice a string	16
`replace`	Swap sub-strings	16
`format(template, args...)`	Interpolate values	16

C.5 Lists

Built-in	Description	See
`append(list, x)`	Add to the end	17
`insert(list, i, x)`	Insert at a position	17
`extend(list, xs)`	Append many	17
`remove(list, x)` / `pop(list)`	Remove by value / by index	17
`sort(list)`, `reverse(list)`	Re-arrange in place	17
`copy(list)`, `clear(list)`	Duplicate / empty	17
`index(list, x)`, `count(list, x)`	Search helpers	17

C.6 Dicts and Sets

Built-in	Description	See
`get(d, k, default?)`, `has(d, k)`	Read and test keys	18
`update(d, dict)`, `keys(d)`, `values(d)`, `items(d)`	Mutate and enumerate	18
`set(x)`	Build a set from a list or string	18
`set_add`, `set_remove`, `set_contains`	Set membership and mutation	18
`set_union`, `set_intersection`, `set_difference`	Set algebra	18

C.7 Random

Built-in	Description	See
`rand()`	Float in `[0, 1)`	19
`randint(a, b)`	Integer in `[a, b]`	19
`seed(n)`	Reproducible sequences	19
`shuffle(list)`	Random re-ordering	19

C.8 Files and OS

Built-in	Description	See
`read_file(path)`, `write_file(path, text)`	Text file I/O	20
`file_exists(path)`	Test for a file	20
`import_file(path)`	Run another script	20

C.9 JSON

Built-in	Description	See
`json_parse(text)`	Decode JSON into values	21
`json_stringify(value)`	Encode values as JSON	21

C.10 Time and Web

Built-in	Description	See
`now()`, `sleep(secs)`	Current time; pause	22
`strftime(pattern, ts?)`	Format a timestamp	22
`http_server(port, html?)`, `http_start(srv)`	Serve a web page	23
`http_request(url)`	Fetch a resource	23

C.11 Machine Learning

Built-in	Description	See
`tensor(list)`, `matrix(list)`	Create tensors and matrices	24
`matmul(a, b)`	Matrix multiplication	24
`sigmoid(x)`, `relu(x)`	Activation functions	24
`mse(preds, targets)`	Mean squared error	24

Appendix D. Scroll Function Index

Every function exported by the fourteen standard scrolls (Part IV). Namespaced access is `scroll.function`; flat access works as well.

Scroll	Functions
`text`	is_blank, count_substring, extract_digits, split_words, slugify, snake_case, kebab_case, camel_case, pascal_case, mask, abbreviate, truncate_words, pluralize, indent, normalize_spaces, initials, rot13, caesar, sort_chars, is_anagram, is_isogram, shuffle_chars, reverse_words, contains_any, count_vowels, count_consonants
`strings`	reverse, is_palindrome, count_words, repeat, truncate, capitalize_words, strip_vowels, remove_spaces, words, has_letter, titleize
`stats`	mean, median, mode, weighted_mean, geometric_mean, harmonic_mean, trim_mean, population_variance, sample_variance, variance, population_stdev, sample_stdev, stdev, range_stat, midrange, mean_absolute_deviation, percentile, quartile1, quartile3, interquartile_range, zscore, standardize, normalize, covariance, correlation
`seq`	head, tail, init, last_item, take, drop, nth, zip, enumerate, interleave, sliding_window, consecutive_pairs, product, frequency, repeat_times, cycle, accumulate, differences, intersperse, shuffle_copy
`lists`	flatten, unique, reversed, chunk, rotate, median, first, last, sum_squares, count_occurrences
`numbers`	is_numeric, is_integer, is_whole, digit_count, digits, from_digits, reverse_digits, to_binary, to_octal, to_hex, from_base, from_binary, from_octal, from_hex, percent, percent_change, round_to, ordinal, with_commas, is_power_of_two, count_bits
`math`	log2, log10, hypot, degrees, radians, sign, clamp, lerp, is_even, is_odd, factorial, gcd, lcm, is_prime, avg, midpoint, digit_sum
`fs`	exists, read, write, append_text, read_lines, write_lines, count_lines, first_line, last_line, file_size, is_empty_file, basename, dirname, ext, join_path, with_extension, copy_text, word_count
`dates`	unix, today, time_str, stamp, fmt, date_parts, add_seconds, add_minutes, add_hours, add_days, add_weeks, add_months, add_years, seconds_between, minutes_between, hours_between, days_between, mod, day_of_week, day_name, day_name_full, month_name, month_name_full, is_weekend, is_weekday, start_of_day, end_of_day, start_of_hour, start_of_minute, is_leap_year, month_days, day_of_year, iso_week, human_duration
`sets`	from_list, from_string, singleton, union, intersection, difference, symmetric_difference, has_item, add_item, remove_item, size, is_empty, is_subset, is_superset, is_disjoint, equals, has_any, has_all, add_all, remove_all
`json`	parse, stringify, load, save, pretty
`os`	get_env, current_dir, program_args, file_read, file_write
`random`	random, int_between, choice, choices, sample, coin_flip
`time`	sleep_ms, epoch, stamp, stamp_utc, today, clock_time